

CloudHUB Build Report

Expanding my private productivity workspace: hybrid storage with TrueNAS, a locked-down AWS rebuild, Terraform, a private AI assistant, and a Discord bot with alerts. Every step includes the screenshots, commands and results, including what broke.

Author: Tayo (github.com/by-tayo) · **Date:** October 2026

IP addresses, account IDs, hostnames, tokens and the tailnet name are redacted throughout.

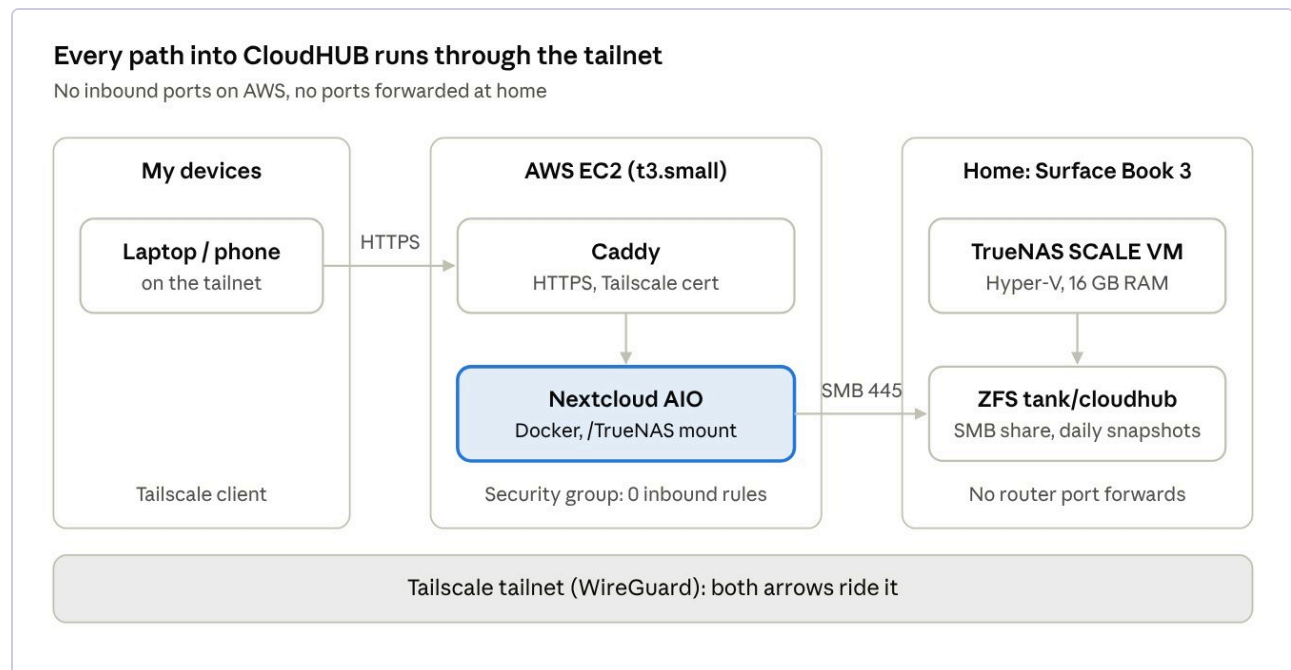


Figure 1. Every path into CloudHUB runs through the tailnet: no inbound ports on AWS, no ports forwarded at home.

Contents

1. Where CloudHUB started
2. Part 1: TrueNAS SCALE on a Surface Book 3
3. Part 2: A fresh, hardened AWS account
4. Part 3: Rebuilding the cloud front door

5. Part 4: Connecting the cloud to home storage
6. Part 5: The port scan surprise
7. Part 6: When "Running" isn't responsive
8. Part 7: Hardening both ends
9. Part 8: Infrastructure as Code with Terraform
10. Part 9: A private AI assistant with Ollama
11. Part 10: A Discord bot for quick actions
12. Part 11: Storage alerts from the home host
13. What I learned
14. Trade-offs
15. Stack
16. Appendix: Setup reference

Where CloudHUB started

CloudHUB is my private productivity workspace: a self-hosted Nextcloud instance on AWS EC2 that nobody on the public internet can reach. Every connection runs through Tailscale, a WireGuard-based VPN, so only my own devices can open it.

This round I set out to do five things:

1. **Add home storage.** Keep Nextcloud in the cloud as the single front door, but put bulk files on a ZFS NAS at home.
2. **Rebuild it properly.** Start over on a fresh, hardened AWS account and lock the server down.
3. **Make it reproducible.** Define the server in Terraform so I can rebuild it with one command.
4. **Add a private AI assistant.** Give Nextcloud AI features backed by a model on my own hardware.
5. **Add quick actions and alerts.** A Discord bot for everyday tasks, plus storage alerts from the home host.

This report walks through all five, including the parts that broke. Those turned out to be where most of the learning happened.

Nextcloud on EC2 is the only front door. My devices reach it over Tailscale, and it reaches the NAS at home the same way, so neither side accepts connections from the open internet.

Part 1: TrueNAS SCALE on a Surface Book 3

My main machine didn't have the free space, so I repurposed a Surface Book 3 (i7-1065G7, 32 GB RAM, 1 TB SSD). It has no Ethernet port, TrueNAS doesn't support Wi-Fi, and it has only one internal drive. So instead of installing TrueNAS on the bare metal, I run it as a **Hyper-V virtual machine** on Windows 11 Pro.

- **VM type:** Hyper-V Generation 2, Secure Boot off, checkpoints off
- **Resources:** 16 GB fixed RAM, 4 vCPUs
- **Disks:** 32 GB boot + 64 GB dynamic data disk
- **Network:** Default Switch (outbound internet)
- **NAS OS:** TrueNAS SCALE / Community Edition 25.10.7

First I set the laptop to never sleep while plugged in and to do nothing when the lid closes. Then I built the VM with a PowerShell script that checks everything before creating it: the ISO exists, the name isn't taken, the switch is there, and there's enough free disk.

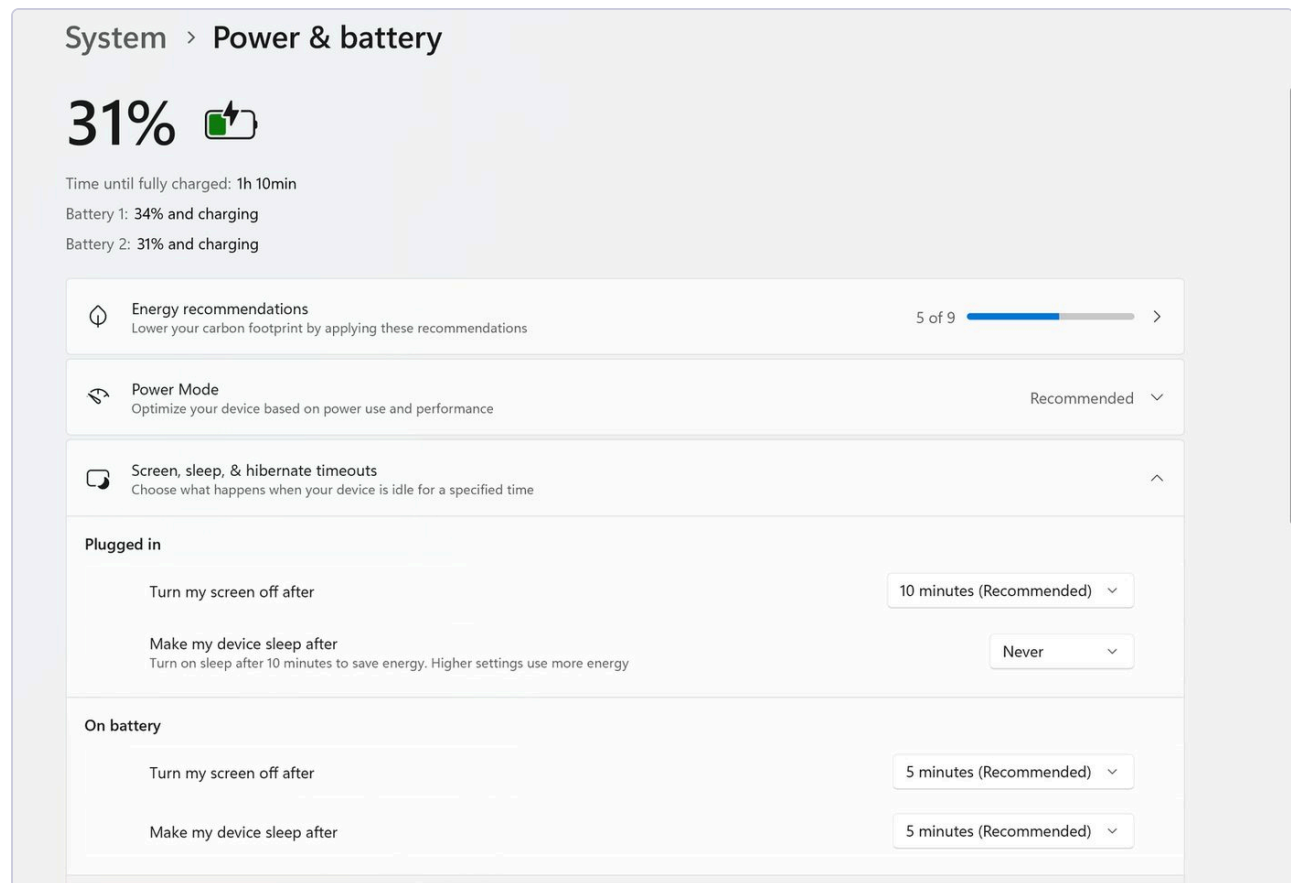


Figure 2. The host can't sleep while plugged in, or the NAS goes offline with it.

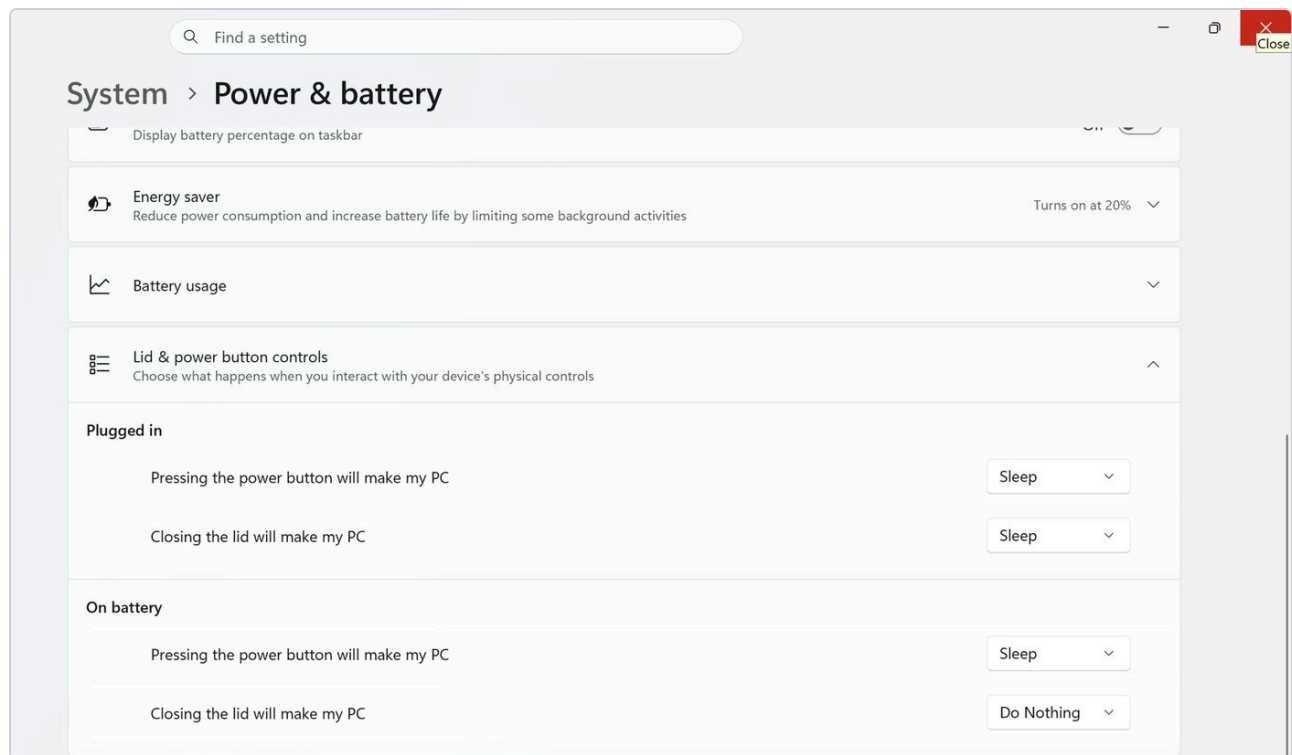


Figure 3. Lid and power button settings: this is where closing the lid gets set to Do Nothing, so the laptop can sit closed and keep serving.

Before building anything, I checked that this Windows edition could run Hyper-V at all (Home editions can't):

```
Windows edition: Microsoft Windows 11 Pro
Hyper-V is already enabled. Skip the restart and go to Phase 0c.
```

The pre-flight check paid off on the first run. The script refused to continue because the host had only 131 GB free, which wasn't enough for the 256 GB data disk I had planned. I dropped the data disk to 64 GB. It's dynamic, so it only grows as it fills.

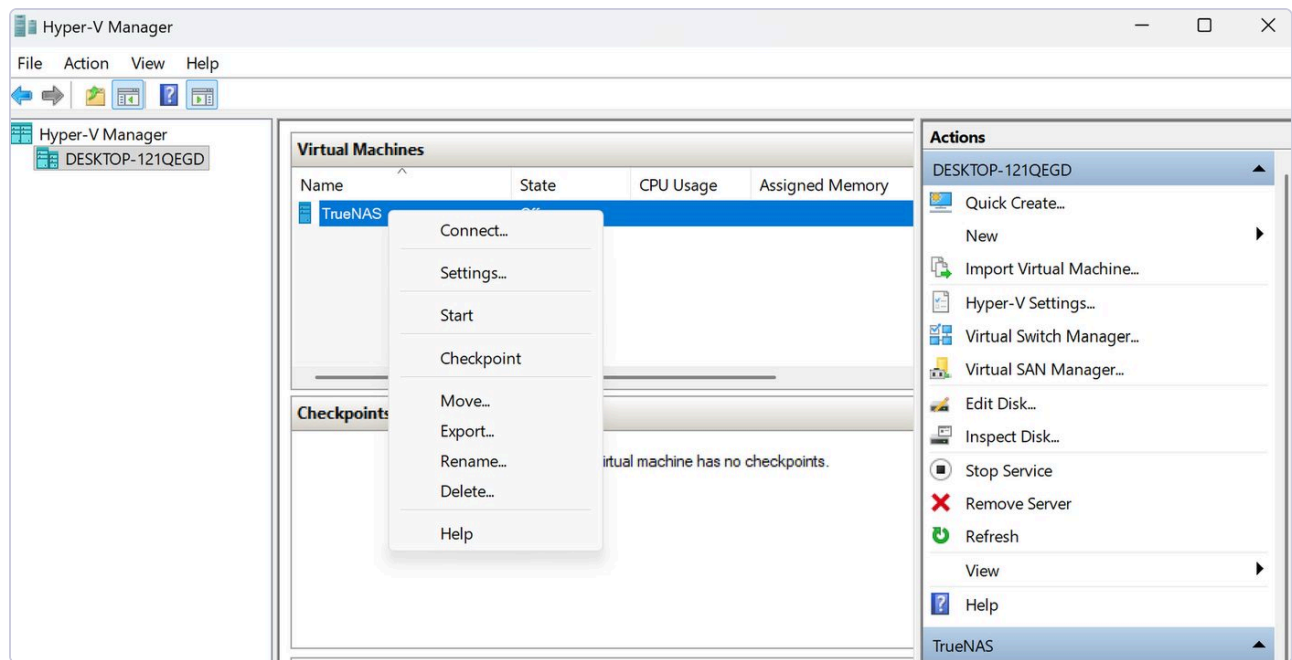


Figure 4. The TrueNAS VM in Hyper-V Manager on the Surface Book 3.

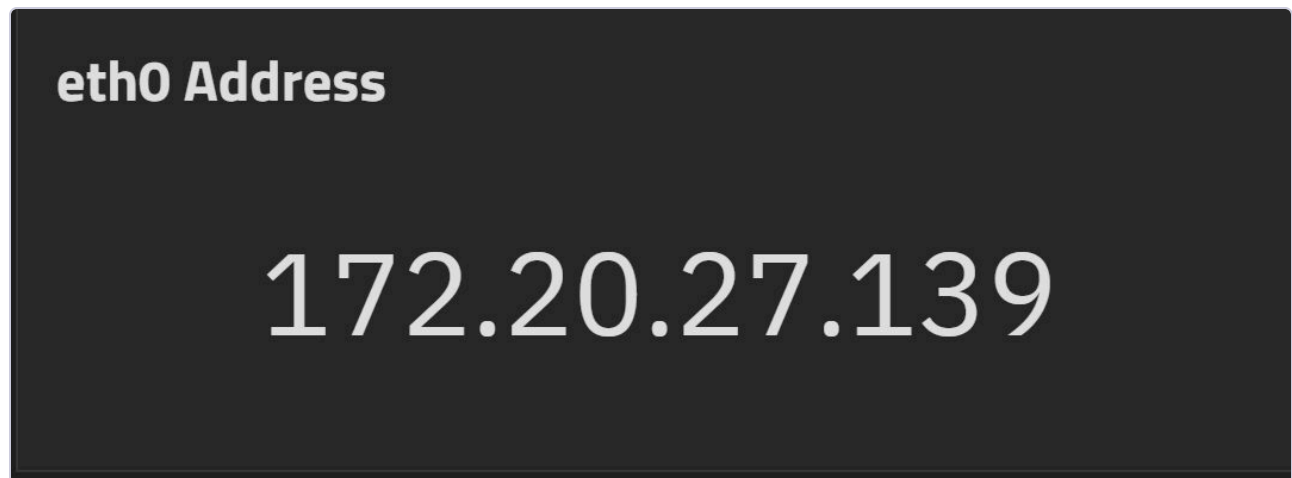


Figure 5. After install, the console shows the address the VM got from Hyper-V's Default Switch (a private, internal address).

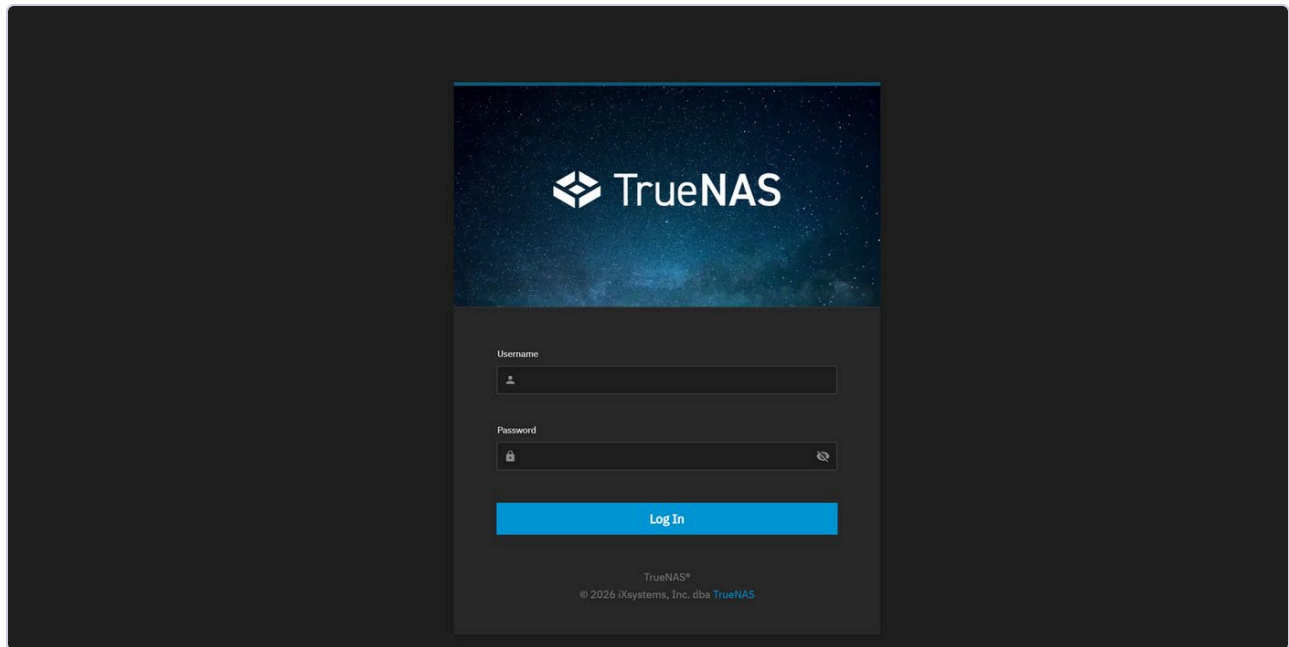


Figure 6. First login to the TrueNAS web UI from the host.

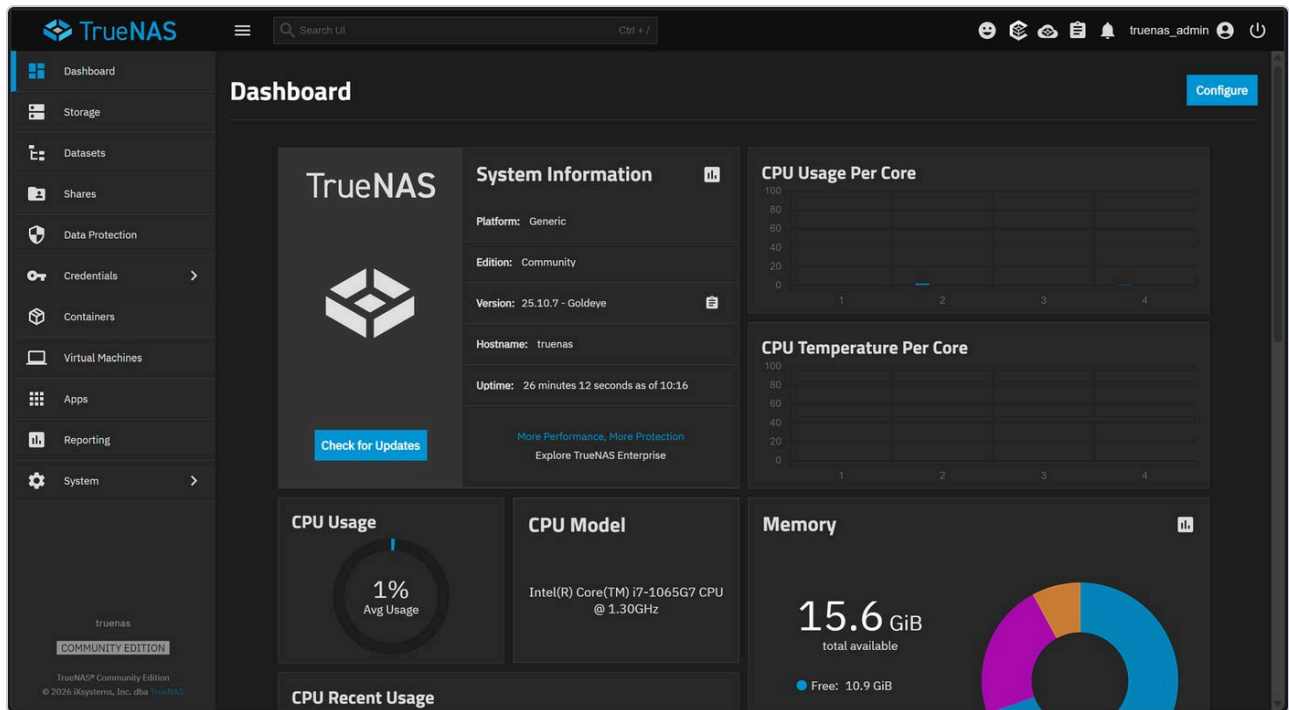


Figure 7. TrueNAS SCALE 25.10.7 up and running inside the VM.

Inside TrueNAS I set up:

- **A ZFS pool** named `tank`, with a dataset `tank/ccloudhub` using the SMB preset. It's a single-disk stripe, so it's fine for a lab but has no redundancy.

- **A dedicated user** `nextcloud-smb`, which has SMB access only: no shell and no admin rights. It has Modify permission on that one dataset.
- **An SMB share** named `cloudhub`.
- **The Tailscale app** from the community train, so the NAS joins the same tailnet as the cloud server.

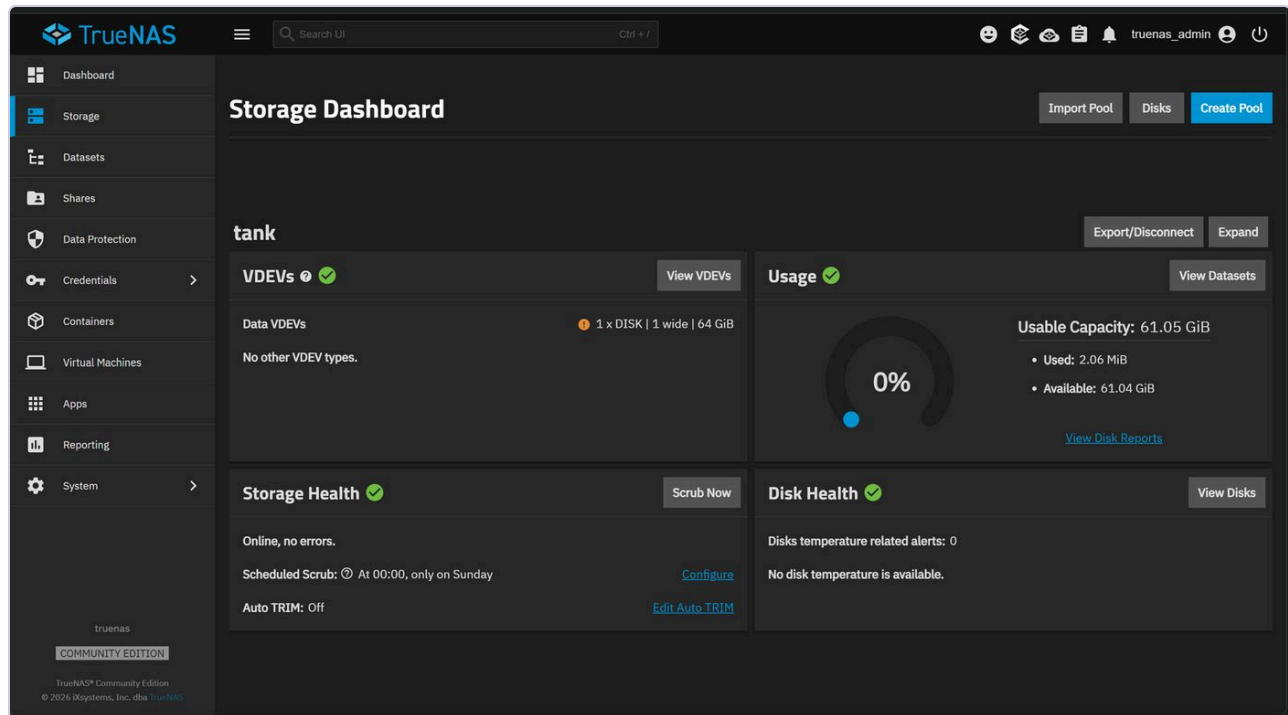


Figure 8. Pool `tank`: one 64 GiB virtual disk, online, no errors.

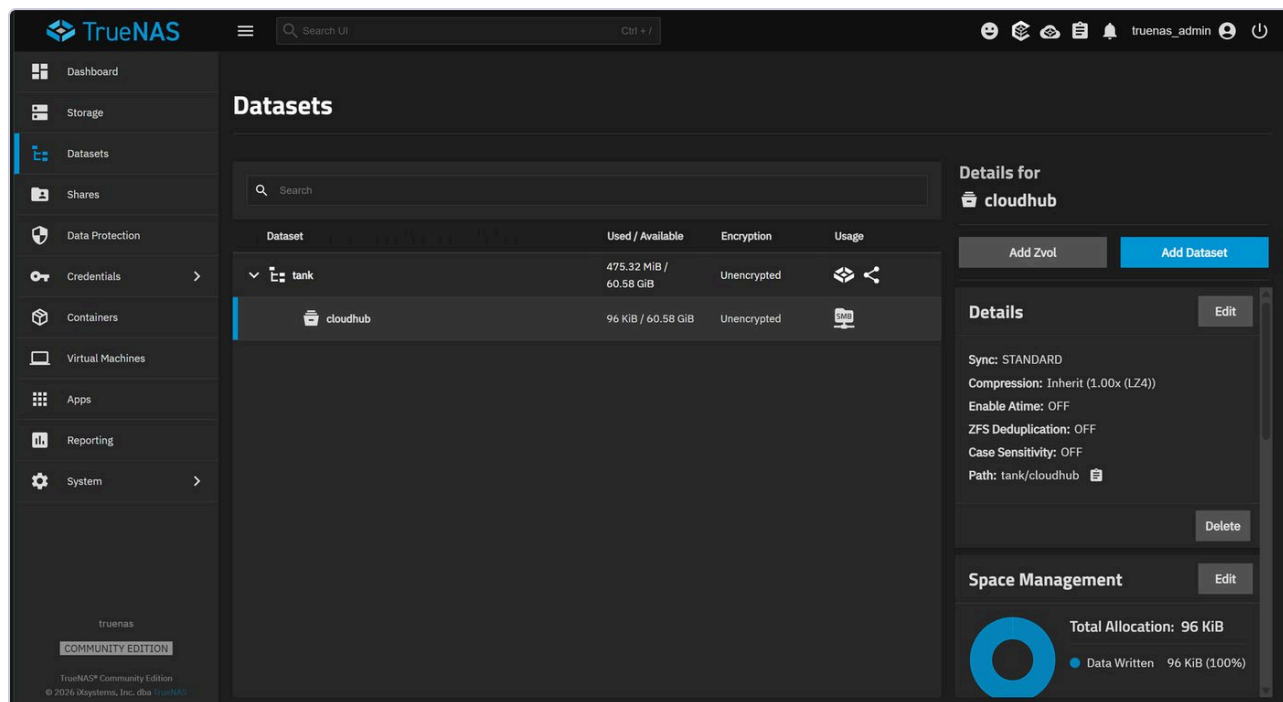


Figure 9. The `tank/cloudhub` dataset: LZ4 compression on, atime off.

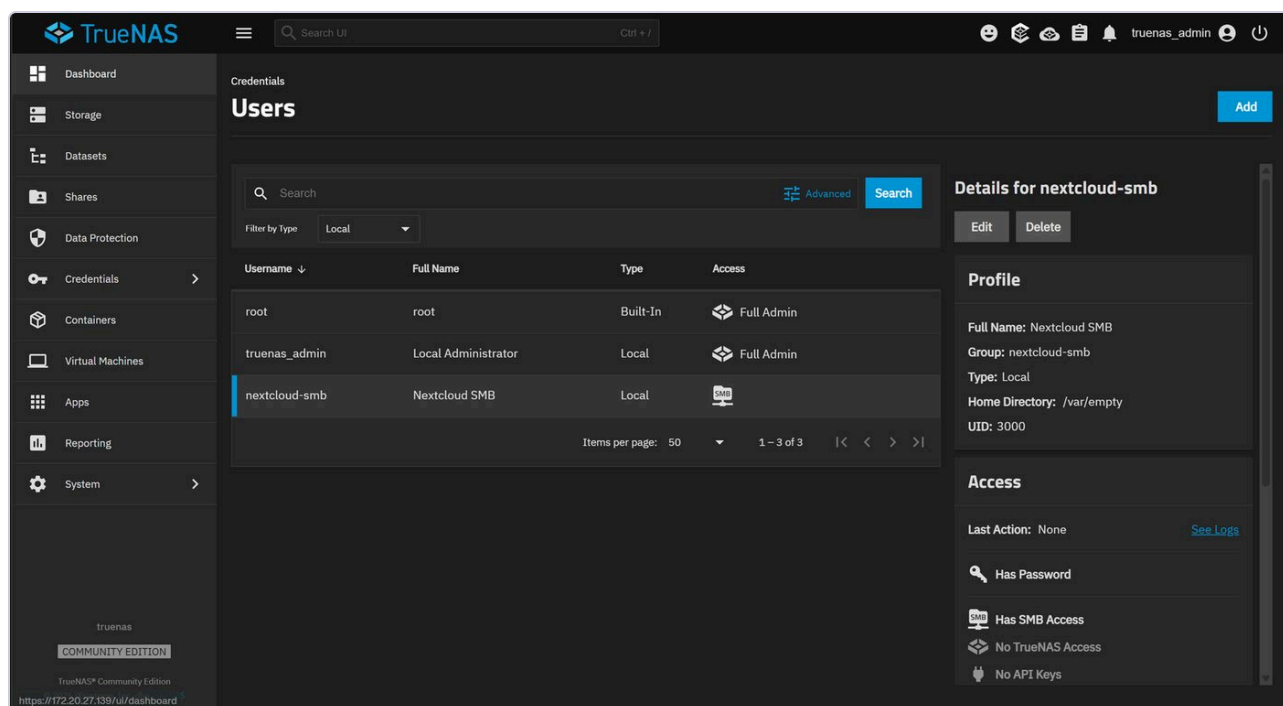


Figure 10. The `nextcloud-smb` service account: SMB access, no TrueNAS access, no API keys.

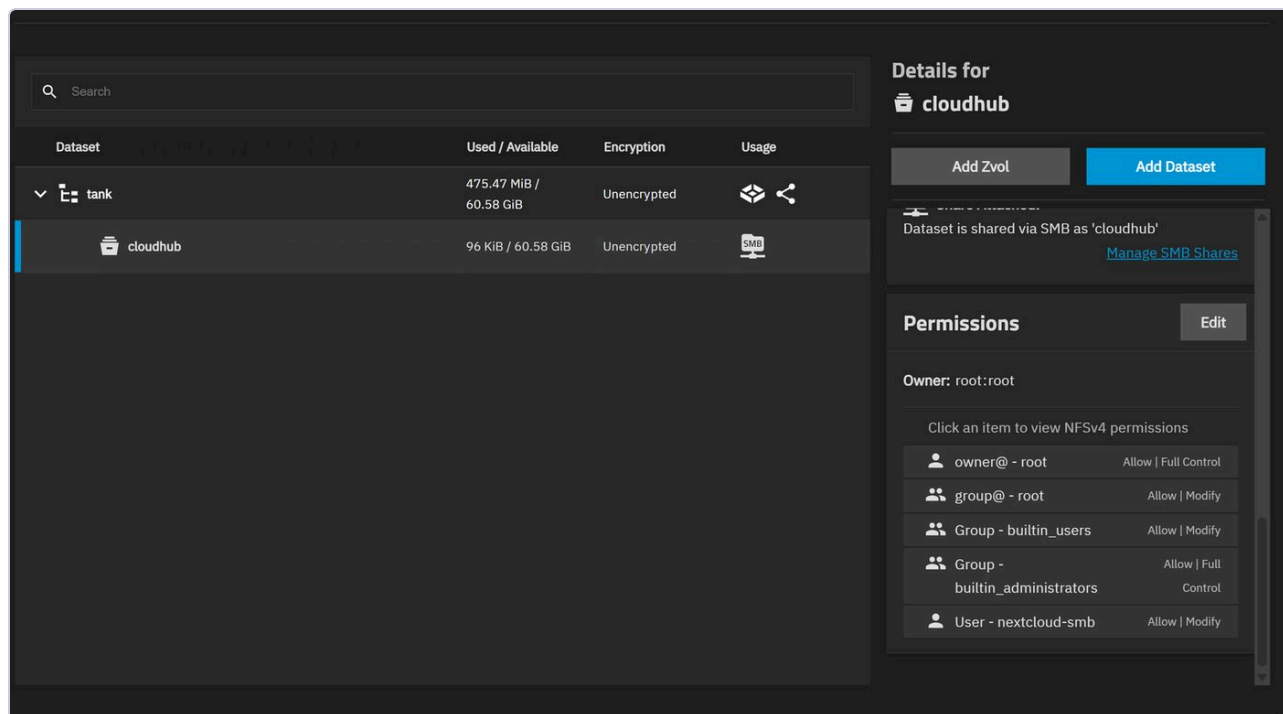


Figure 11. The dataset ACL with `nextcloud-smb` granted Modify. `builtin_users` is still listed here; I removed it during hardening (Part 7).

One catch: an enabled share isn't enough. The `cloudhub` share showed as enabled, but the SMB service itself was stopped. Starting the service, and setting it to start automatically, fixed it.

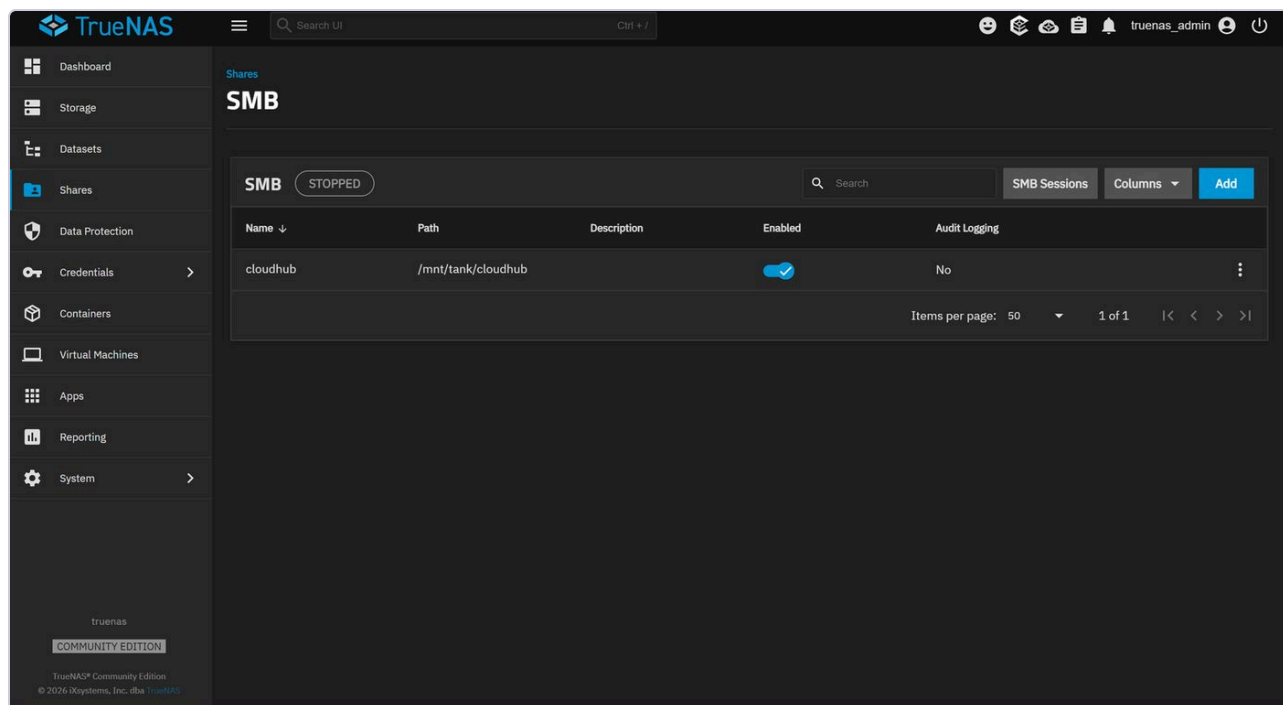


Figure 12. Share enabled, service stopped.

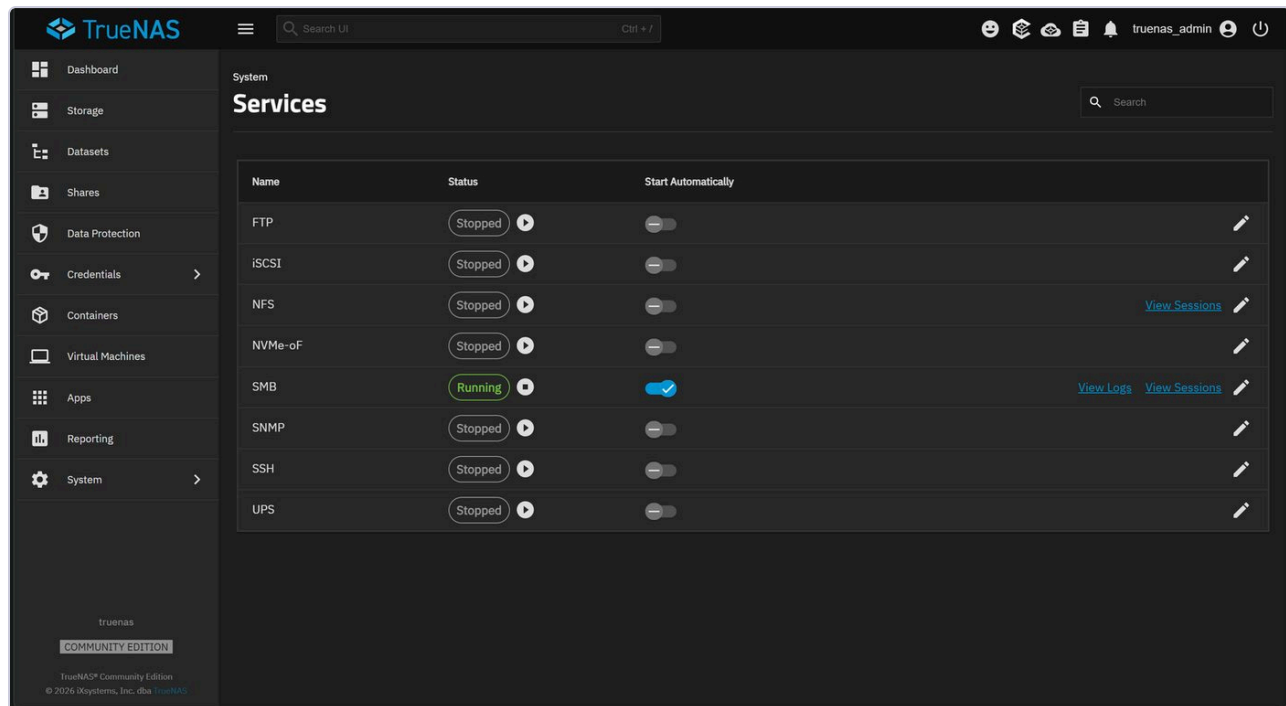


Figure 13. SMB running and set to start at boot.

To test it, I opened the share from Windows Explorer and created a `test.txt` file.

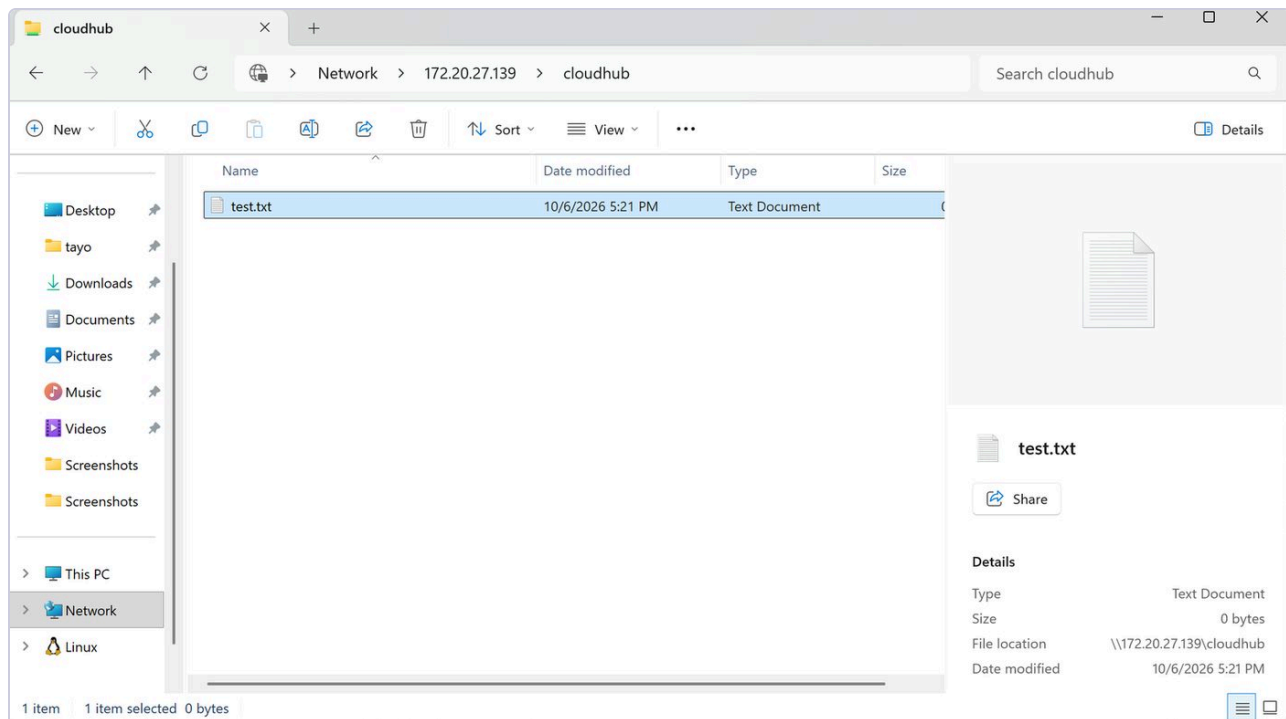


Figure 14. First file on the share, written from the Windows host.

Tailscale needs host networking here. The app's default userspace mode is fine for reaching the TrueNAS web UI. But Nextcloud has to reach the SMB service on the TrueNAS

host itself, so I turned host network **on** and userspace **off**.

If the NAS drops off the tailnet, restart the app first. At one point the app showed Running while the Tailscale console showed the machine as offline. A restart reconnected it.

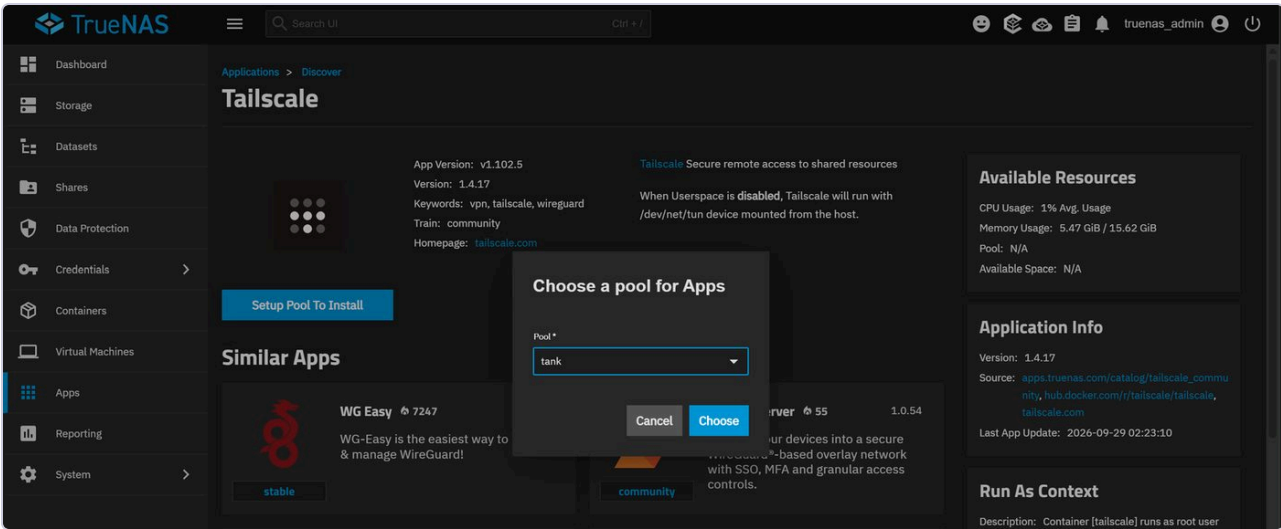


Figure 15. Installing Tailscale from the TrueNAS app catalog. Apps need a pool, so I picked `tank` .

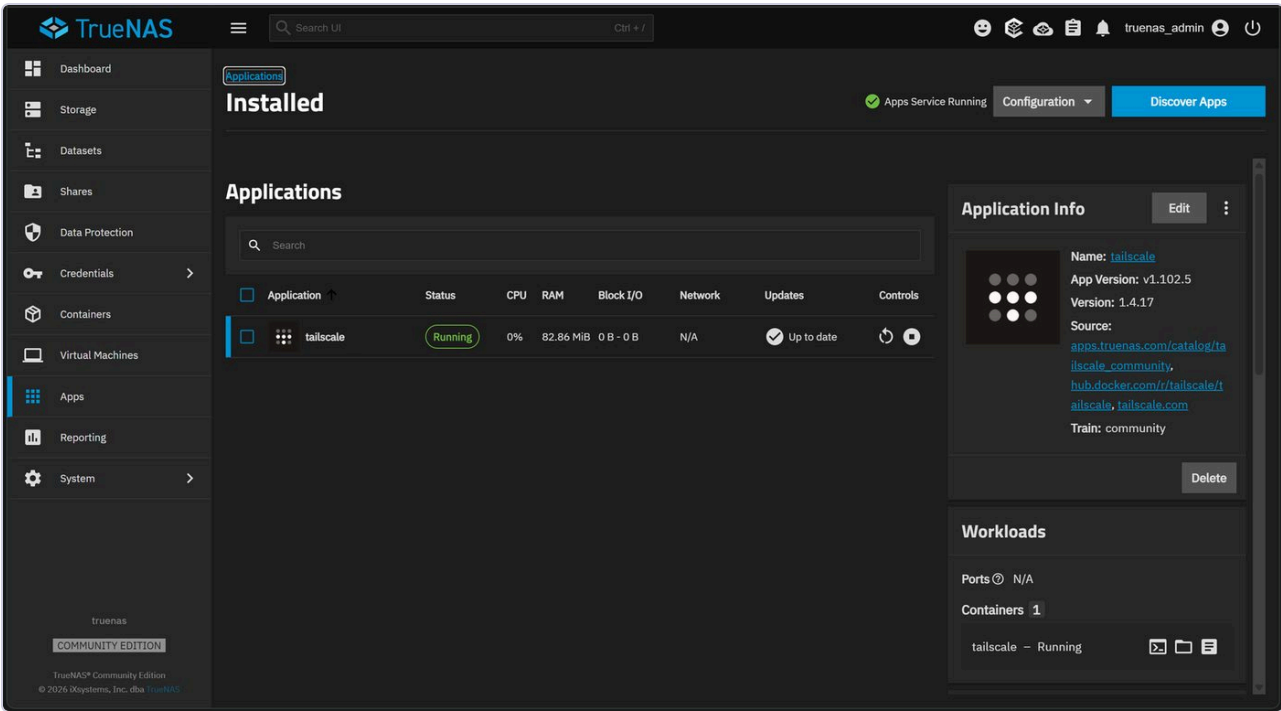


Figure 16. The Tailscale app running in TrueNAS.

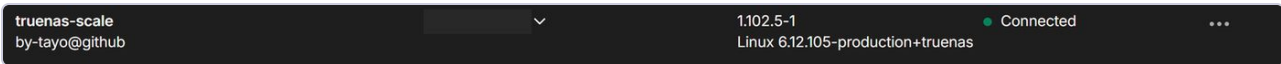


Figure 17. The NAS joins the tailnet as `truenas-scale` .

Figure 18. The moment it dropped off: offline in the console while the app still said Running. Restarting the app fixed it.

Part 2: A fresh, hardened AWS account

I had deleted my old AWS account, so I started a new one and set it up the way I'd want a real account to look on day one:

- **Root account MFA.** After that, I don't use the root account for daily work.
- **An IAM Identity Center admin user** for my everyday logins. It uses short-lived sign-ins through the AWS access portal, not long-lived access keys.
- **A \$25 monthly budget alert**, so a forgotten resource can't quietly run up a bill.
- **One region:** us-east-2 (Ohio).

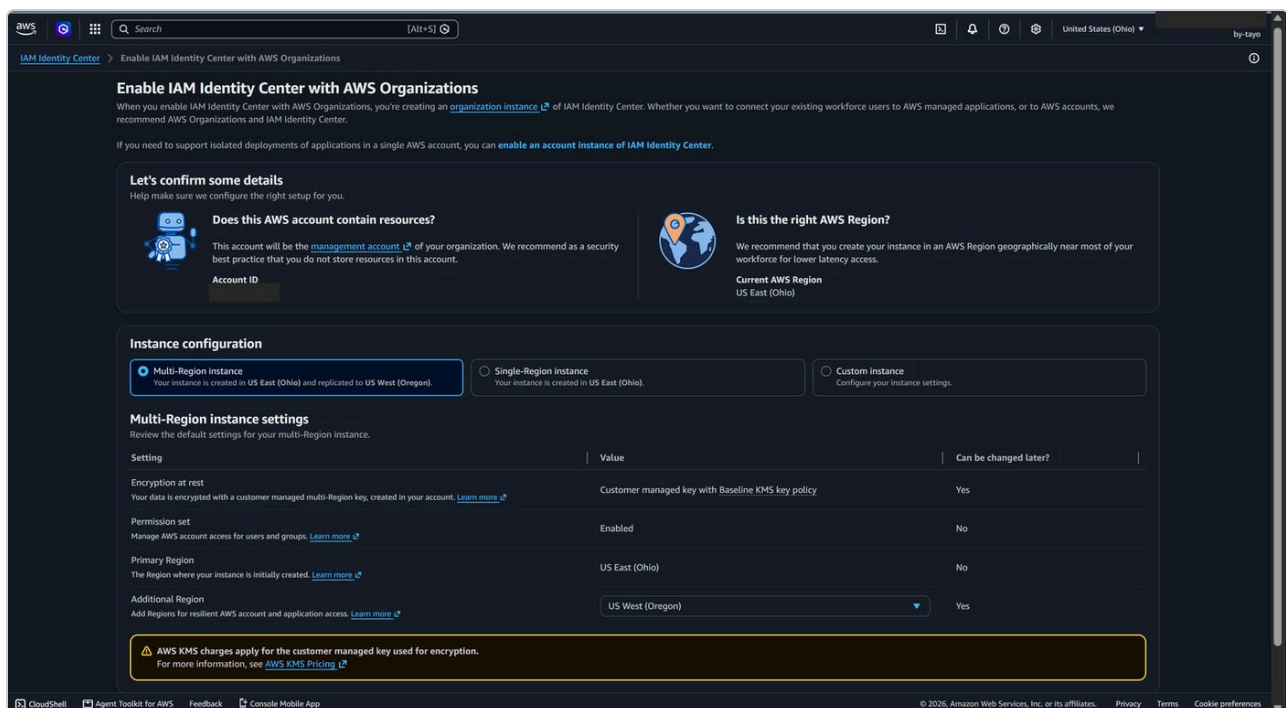


Figure 19. Enabling IAM Identity Center in us-east-2 (account ID redacted).

Part 3: Rebuilding the cloud front door

The server is a t3.small running Ubuntu 24.04 with a 30 GB encrypted gp3 volume. It runs three things: Tailscale, Caddy as a reverse proxy, and Nextcloud All-in-One (AIO) in Docker.

Tailscale, with SSH built in. Joining the tailnet with `--ssh` means I can reach the server without port 22 open and without key files:

```
sudo tailscale up --hostname=cloudhub --ssh
```

Caddy with certificates it renews itself. Tailscale can issue a real HTTPS certificate for the machine's private `*.ts.net` name. Granting Caddy permission lets it fetch and renew that certificate on its own, so there's no manual `tailscale cert` step that expires every 90 days:

```
echo 'TS_PERMIT_CERT_UID=caddy' | sudo tee -a /etc/default/tailscaled  
sudo systemctl restart tailscaled
```

```
cloudhub.<your-tailnet>.ts.net {  
    reverse_proxy localhost:11000  
}
```

Nextcloud AIO behind Caddy. AIO listens only on localhost, so Caddy is the only way in:

```
sudo docker run -d --init --sig-proxy=false \  
  --name nextcloud-aio-mastercontainer --restart always \  
  --publish 8080:8080 \  
  --env APACHE_PORT=11000 \  
  --env APACHE_IP_BINDING=127.0.0.1 \  
  --env SKIP_DOMAIN_VALIDATION=true \  
  --volume nextcloud_aio_mastercontainer:/mnt/docker-aio-config \  
  --volume /var/run/docker.sock:/var/run/docker.sock:ro \  
  nextcloud/all-in-one:latest
```

Two errors along the way looked scary but weren't:

- **DNS_PROBE_FINISHED_NXDOMAIN.** The problem was DNS, not the server. My laptop wasn't on the tailnet, so MagicDNS couldn't resolve the private name. Once the laptop connected, the page loaded.
- **HTTP 502.** This was a good sign. Caddy was up and serving HTTPS, but Nextcloud only starts listening after you finish the AIO setup on port 8080.

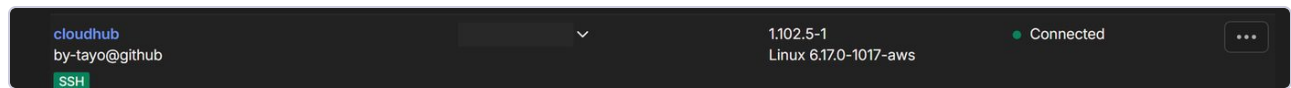


Figure 20. The EC2 server on the tailnet as `cloudhub` , with Tailscale SSH enabled.

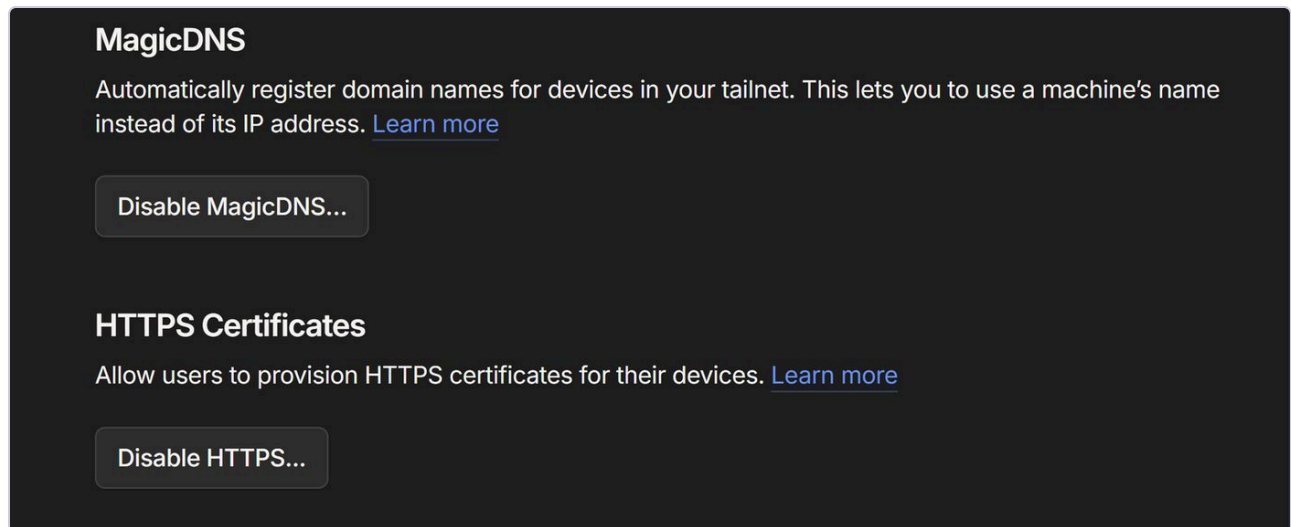


Figure 21. MagicDNS and HTTPS certificates turned on in the Tailscale admin console. This is what lets Caddy get a real certificate for the private name.

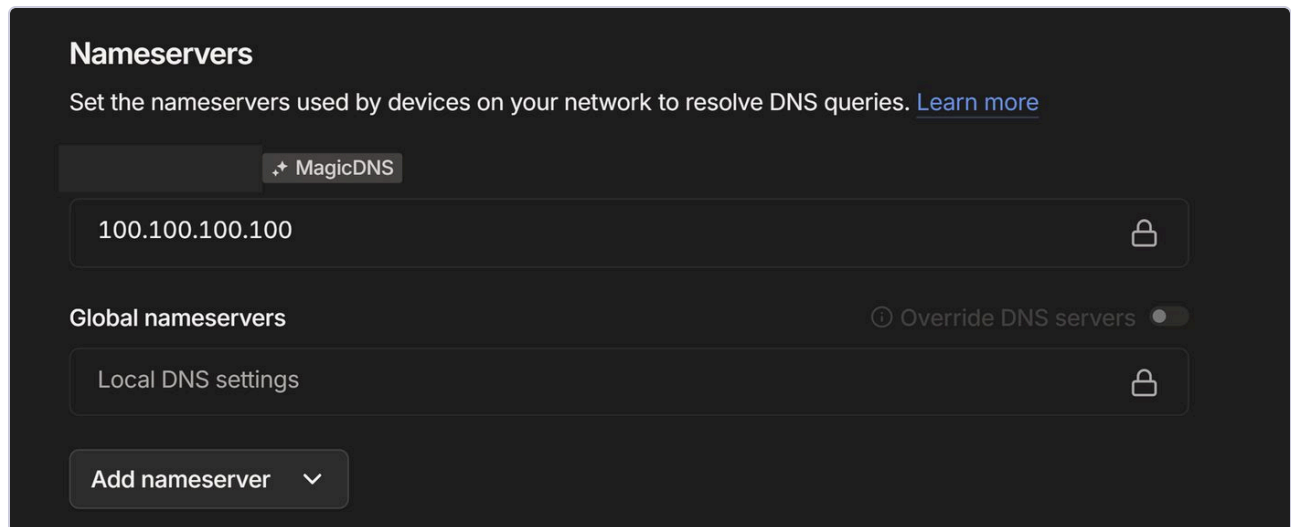


Figure 22. MagicDNS answers at 100.100.100.100, but only for devices that are on the tailnet. That's why the next error happened.

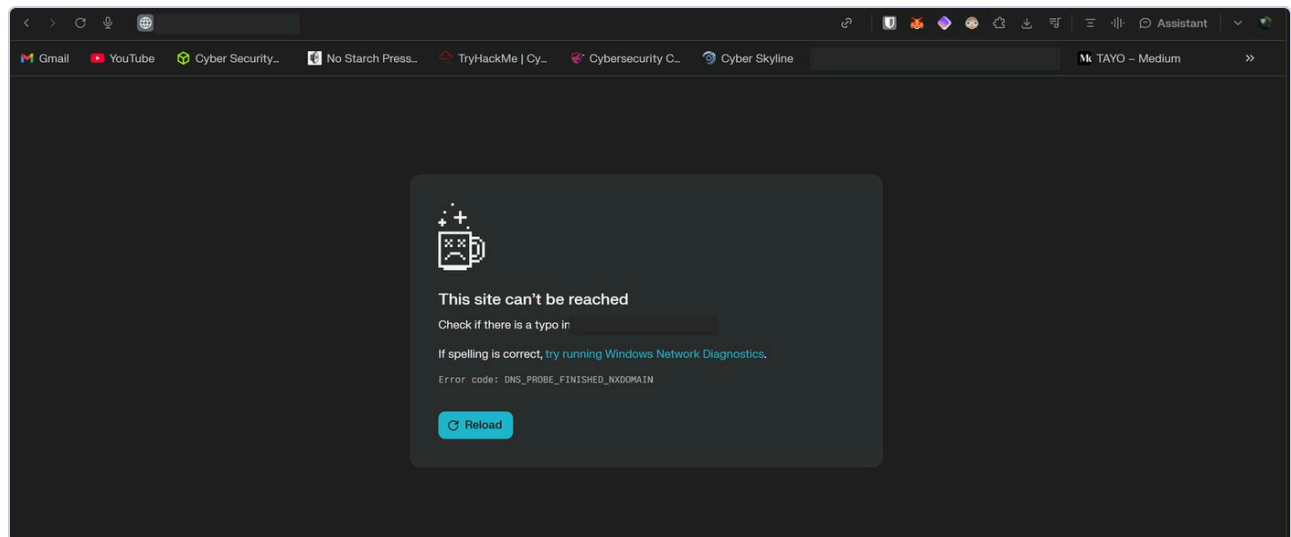


Figure 23. NXDOMAIN: the laptop wasn't on the tailnet yet, so the private name didn't resolve.

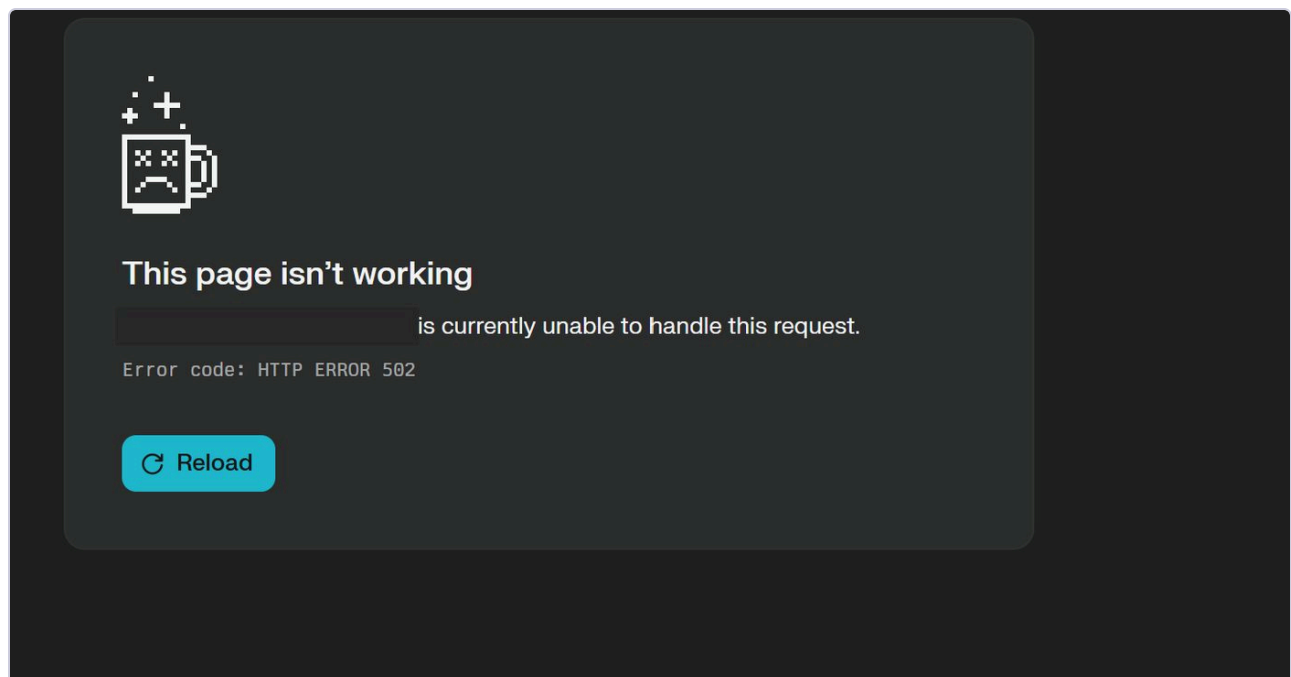


Figure 24. A 502 here was progress: HTTPS worked, and Nextcloud just wasn't set up yet.

Part 4: Connecting the cloud to home storage

The EC2 server and the NAS are on the same tailnet, so Nextcloud reaches the SMB share at the NAS's Tailscale address. **No ports are forwarded on my home router.**

Before touching Nextcloud, I confirmed from EC2 that the NAS answered on the SMB port:

```
tailscale ping truenas-scale
nc -zv <truenas-tailscale-ip> 445
```

Then, in Nextcloud, I enabled the **External storage support** app and added a mount:

- **Folder name:** TrueNAS
- **Backend:** SMB / CIFS
- **Host:** the NAS's Tailscale IP
- **Share:** cloudhub
- **Login:** nextcloud-smb

The first attempt showed an **External mount error** even though every connectivity check passed. On the next check it had resolved, and my test file appeared. It had been saved as `test.txt.txt`, because Windows hides file extensions by default.

To prove it worked in both directions, I created a file on TrueNAS and saw it in Nextcloud. Then I created a `from-cloudhub` file in Nextcloud and saw it on TrueNAS.

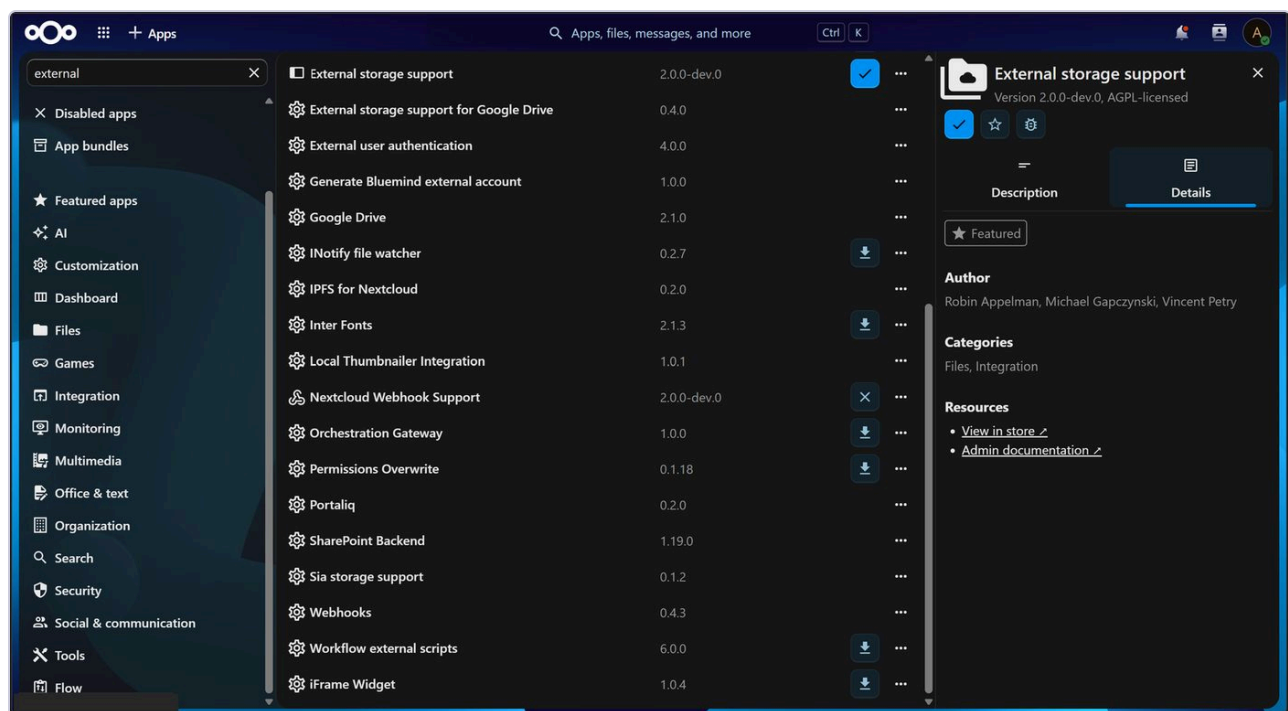


Figure 25. Step one in Nextcloud: enable the External storage support app.

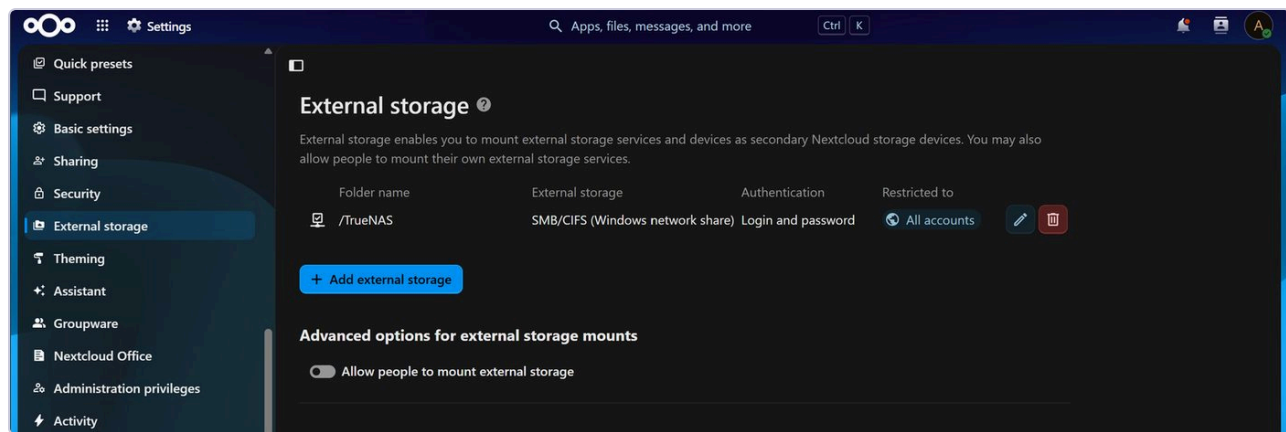


Figure 26. The /TrueNAS mount over SMB/CIFS. It starts out available to all accounts; I restricted it in Part 7.

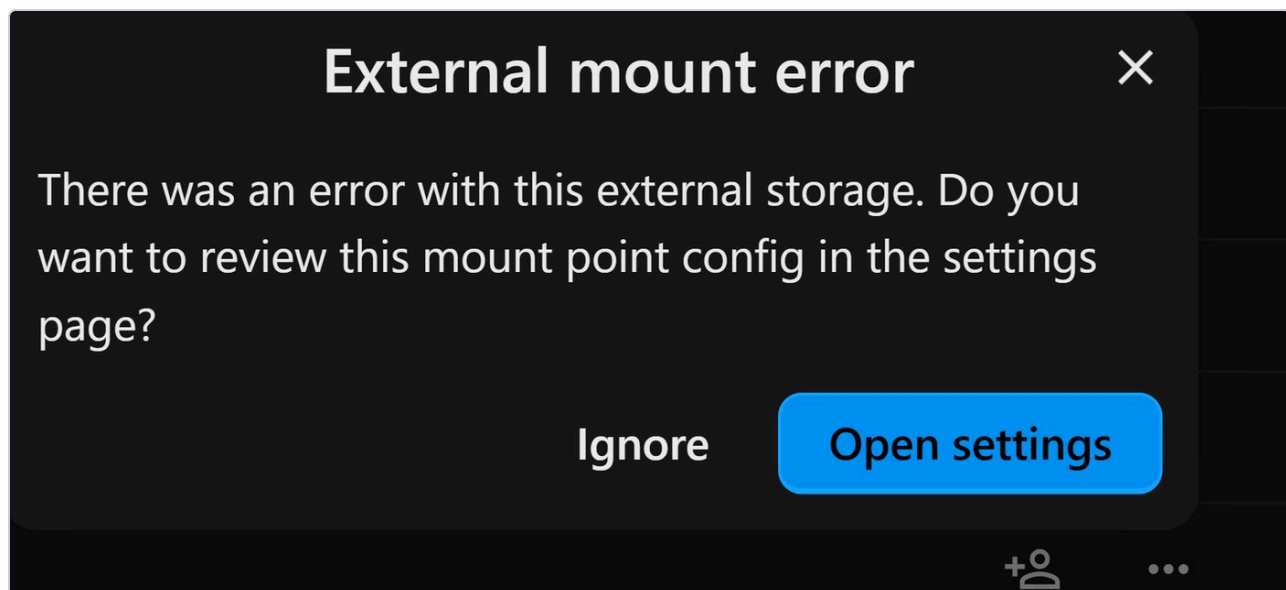


Figure 27. The External mount error that showed up even though the connectivity checks passed.

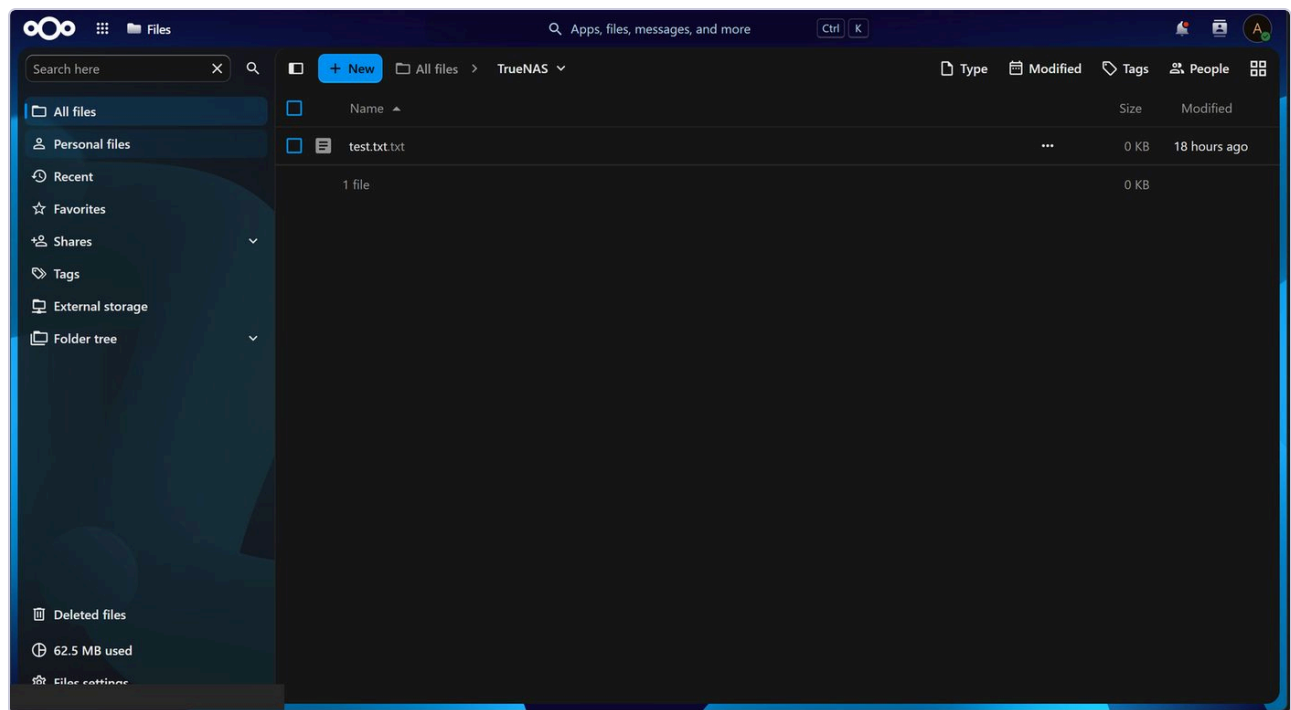


Figure 28. Once the error cleared: my test file, saved by Windows as `test.txt.txt`.

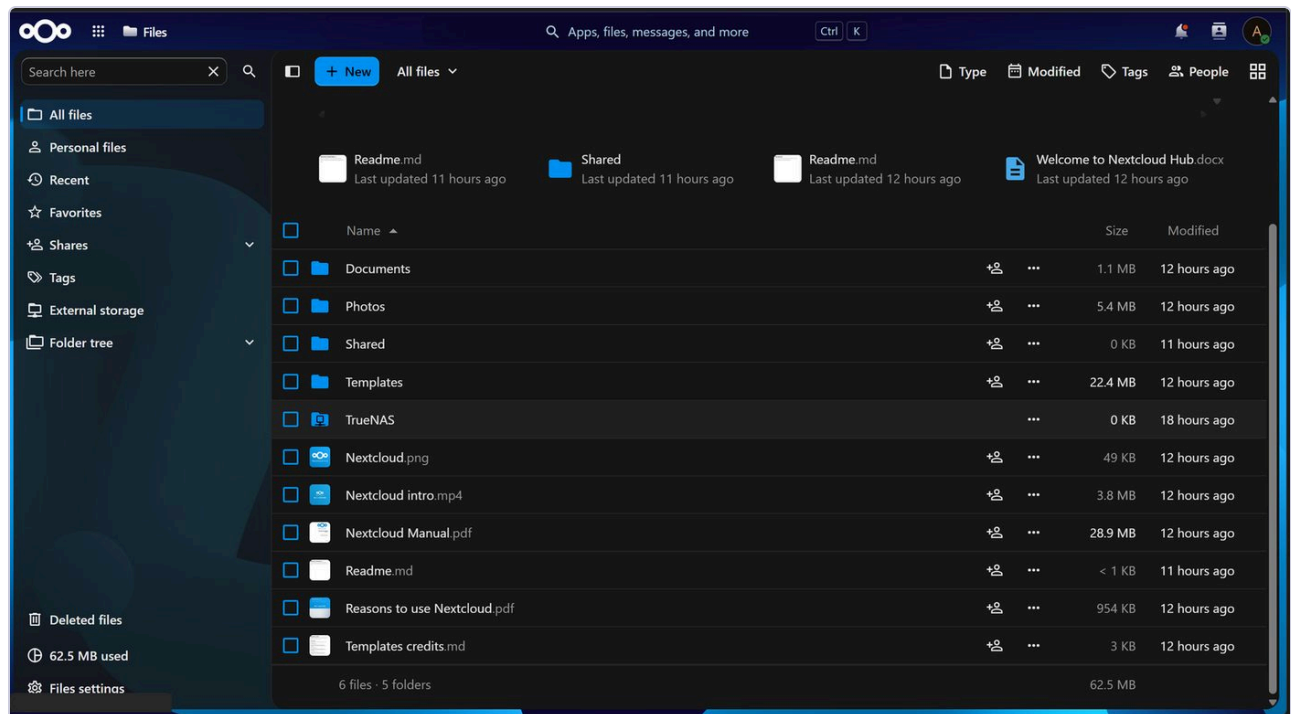


Figure 29. The TrueNAS folder sitting next to my regular Nextcloud files.

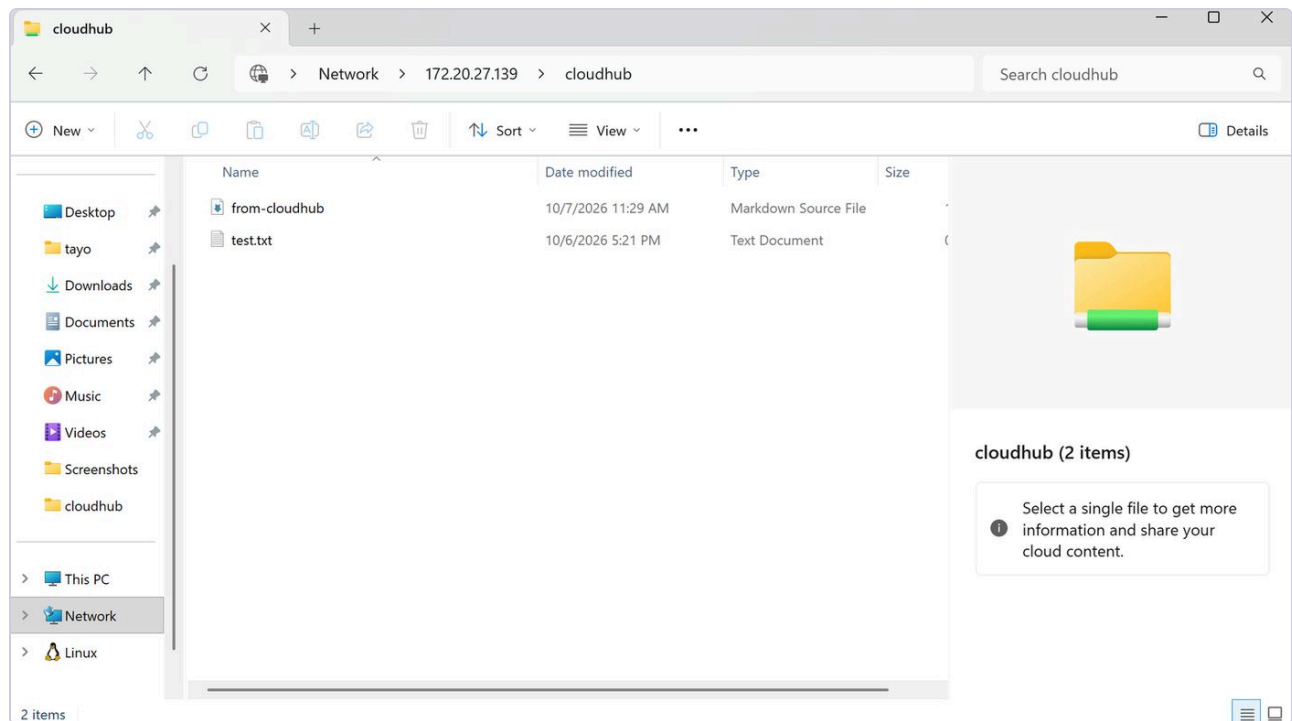


Figure 30. The round trip: `from-cloudhub`, created in Nextcloud on AWS, landed on the NAS at home.

Part 5: The port scan surprise

The design said zero public exposure, but the scan said otherwise. I scanned the instance's public IP from outside the tailnet, and ports **22 (SSH)** and **443 (HTTPS)** came back open.

The cause was that the instance had been launched with the console wizard's default security group, `launch-wizard-1`, not the locked-down one I'd planned. The EC2 console listed the group by its ID, not a friendly name, which made it easy to miss.

The fix was to delete every inbound rule. That works because Tailscale makes **outbound** connections to its coordination servers and relays, so nothing needs to come in.

- **Inbound rules:** SSH 22 and HTTPS 443 before → none after
- **Outbound rules:** all traffic, unchanged
- **Rescan of the public IP:** 22 and 443 open before → 22, 80, 443, 8080, and 8443 all closed after
- **Nextcloud and SSH over Tailscale:** still working

Lesson: verify the security group you actually got, not the one you meant to create.

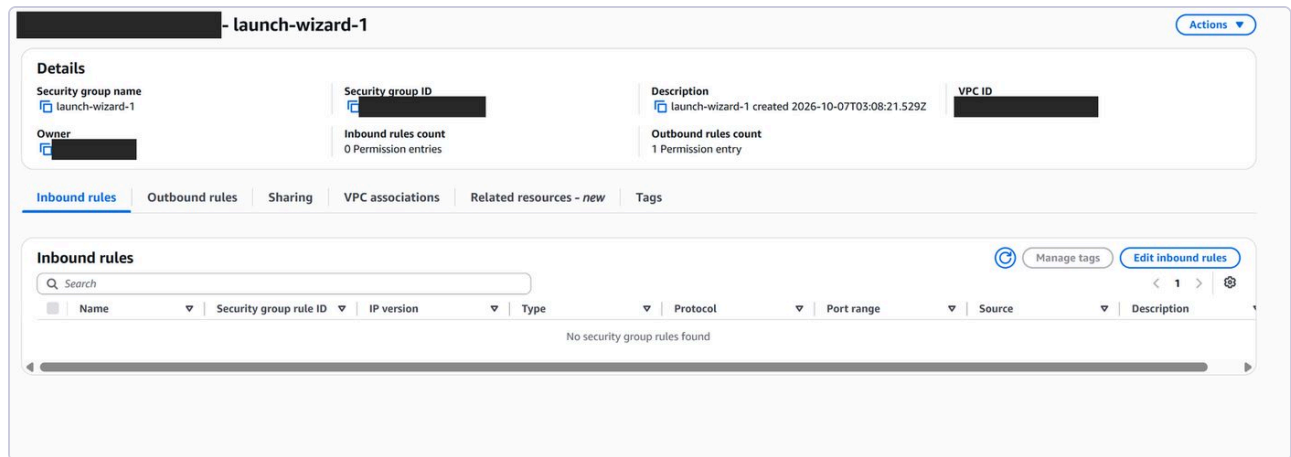


Figure 31. After the fix: `launch-wizard-1` with 0 inbound rules (IDs redacted).

Part 6: When "Running" isn't responsive

After the security group change, the instance still showed **Running** in AWS but stopped answering over Tailscale. Pages timed out with `ERR_CONNECTION_TIMED_OUT`, and so did SSH.

A reboot from the EC2 console brought it back. Right after boot, `free -h` showed why it had likely stalled: **1.6 GB of 1.9 GB** RAM was already in use. After restarting the containers with 2 GB of swap on, usage dropped to **1.1 GB, with 737 MB available**.

The t3.small's 2 GB of RAM is tight for Nextcloud AIO with Nextcloud Office and Talk running. I added a 2 GB swap file as a buffer:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

If memory gets tight again, the next options are turning off Office and Talk in AIO or moving to a t3.medium.

One smaller fix on my laptop: `ssh cloudhub` timed out from Windows because the short name didn't resolve there. An entry in `~/.ssh/config` pointing `cloudhub` at its Tailscale IP fixed it.



This site can't be reached

took too long to respond.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)
- [Running Windows Network Diagnostics](#)

Error code: ERR_CONNECTION_TIMED_OUT

[↻ Reload](#)

[Learn more](#)

Figure 32. What it looked like: AWS said Running, and the browser said otherwise.

Part 7: Hardening both ends

Once everything worked, I tightened it up:

- **Daily ZFS snapshots.** A periodic snapshot task on `tank/cloudhub` runs daily at 00:00 and keeps snapshots for 2 weeks. I also took a manual test snapshot to confirm it worked. If a file gets deleted or encrypted, I can roll back to yesterday.
- **Least privilege on the dataset.** I removed the default `builtin_users` entry from the dataset's ACL, so only `nextcloud-smb` can write to it.
- **Least privilege in Nextcloud.** The external mount was visible to every Nextcloud account. I restricted it to the admin account:

```
sudo docker exec --user www-data nextcloud-aio-nextcloud \  
php occ files_external:applicable --add-user=admin 1
```

Here `1` is the mount's ID, which `occ files_external:list` shows.

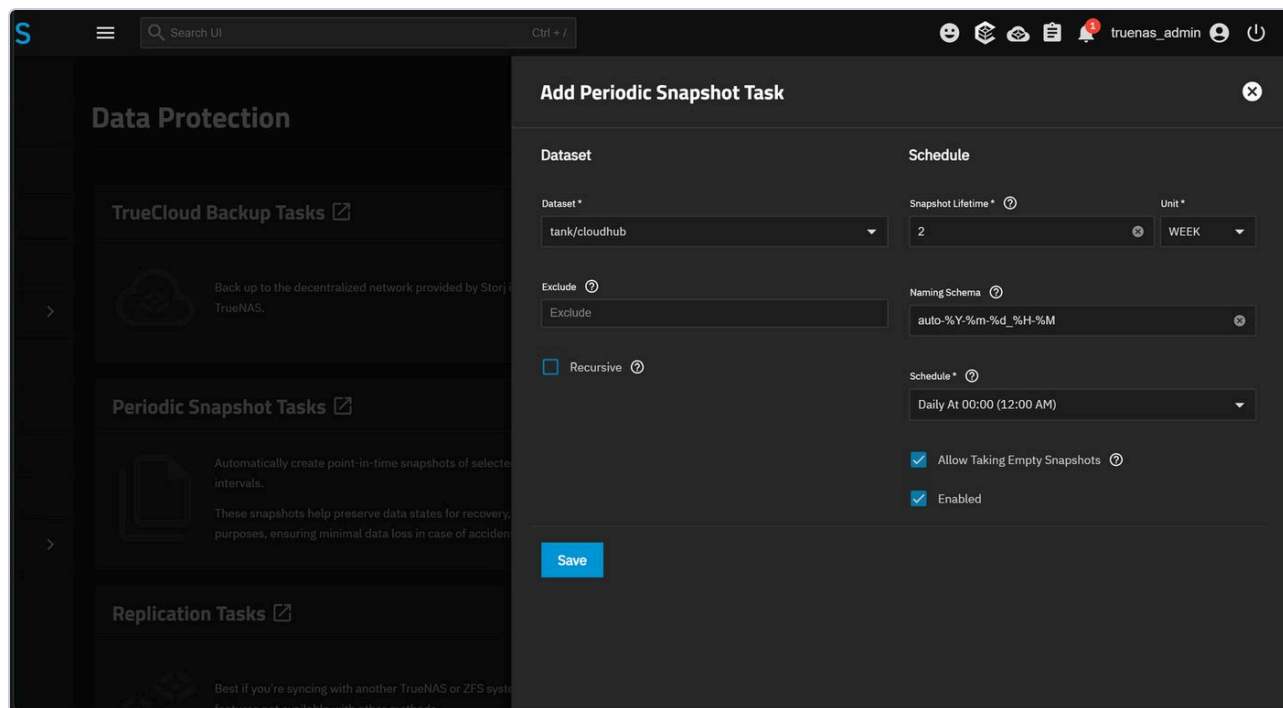


Figure 33. The daily snapshot task: `tank/cloudhub` , 00:00, kept for 2 weeks.

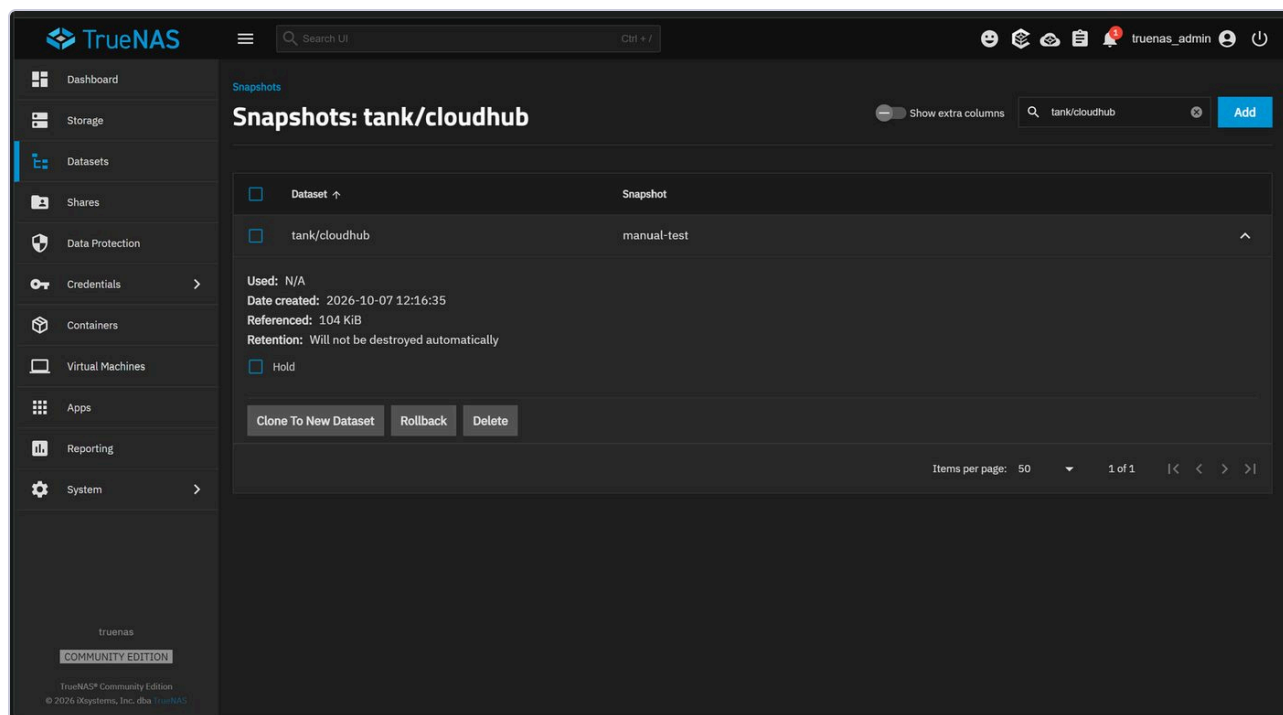


Figure 34. The `manual-test` snapshot, with Rollback one click away.

Part 8: Infrastructure as Code with Terraform

The port scan surprise happened because I built the server by clicking through the console. So the next step was to define the server in code, where the secure settings are written down and can't be skipped by accident.

The `terraform/` folder builds the EC2 side from scratch:

- **Base image:** the latest Ubuntu 24.04 AML.
- **Security group:** **no inbound rules**, outbound only.
- **Disk:** an encrypted 30 GB gp3 volume.
- **Instance settings:** IMDSv2 required, and **no SSH key pair**. Admin access is Tailscale SSH only.
- **A cloud-init script** that installs swap, Docker, Tailscale, Caddy with its certificate permission, and the Nextcloud AIO container, logging to `/var/log/cloudhub-setup.log`.

Signing in with SSO, not access keys. Terraform uses my IAM Identity Center login through an AWS CLI profile, so there are no long-lived keys sitting on my laptop:

```
aws configure sso          # session: cloudhub-sso, region: us-east-2, profile: cloudhub
aws sts get-caller-identity --profile cloudhub
```

Handling the one secret. The Tailscale auth key goes into `terraform.tfvars`, which is git-ignored and never committed. Because the key ends up in the instance's user data, I made it **single-use, ephemeral, and set to expire in a day**.

Then I ran a test deploy with a separate name, `cloudhub-tf`, so it couldn't touch the live server:

```
terraform init
terraform plan -out cloudhub-tf.plan # preview: a security group and an EC2 instance
terraform apply cloudhub-tf.plan
```

The checks on the fresh instance:

- **Inbound rules on the new security group:** `[]`, none
- **Tailscale admin console:** `cloudhub-tf` Connected, tagged Ephemeral and SSH
- **cloud-init status:** `done`
- **Setup log:** ended with `CloudHUB setup finished`
- **Swap:** 2 GB `/swapfile` active

- **New HTTPS URL:** 502, as expected before the AIO setup
- **AIO login page on port 8080:** loaded over Tailscale

The real output from the test instance. Its security group had no inbound rules:

```
PS> aws ec2 describe-security-groups --group-ids <sg-id> --profile cloudhub --query "SecurityGroups[0].IpPermissions"
[]
```

And over Tailscale SSH, cloud-init had finished on its own:

```
ubuntu@ip-172-31-x-x:~$ cloud-init status
status: done

ubuntu@ip-172-31-x-x:~$ sudo tail -n 3 /var/log/cloudhub-setup.log
Status: Downloaded newer image for nextcloud/all-in-one:latest
5d1a37a99bc7022e9d2dc97363aee98c1dbf1ecb89f17c5b6525d54ad19bf96d
=== CloudHUB setup finished 2026-10-08T00:19:52+00:00: https://cloudhub-tf.<tailnet>.ts.net ===

ubuntu@ip-172-31-x-x:~$ swapon --show
NAME      TYPE  SIZE  USED  PRI0
/swapfile file   2G   76K   -1
```

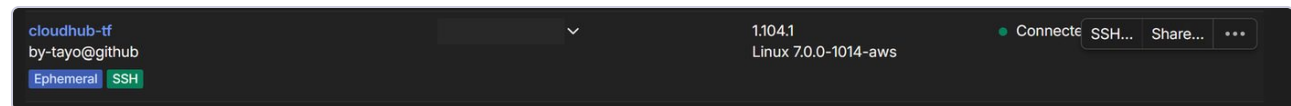


Figure 35. The test machine joined the tailnet by itself, tagged Ephemeral and SSH.

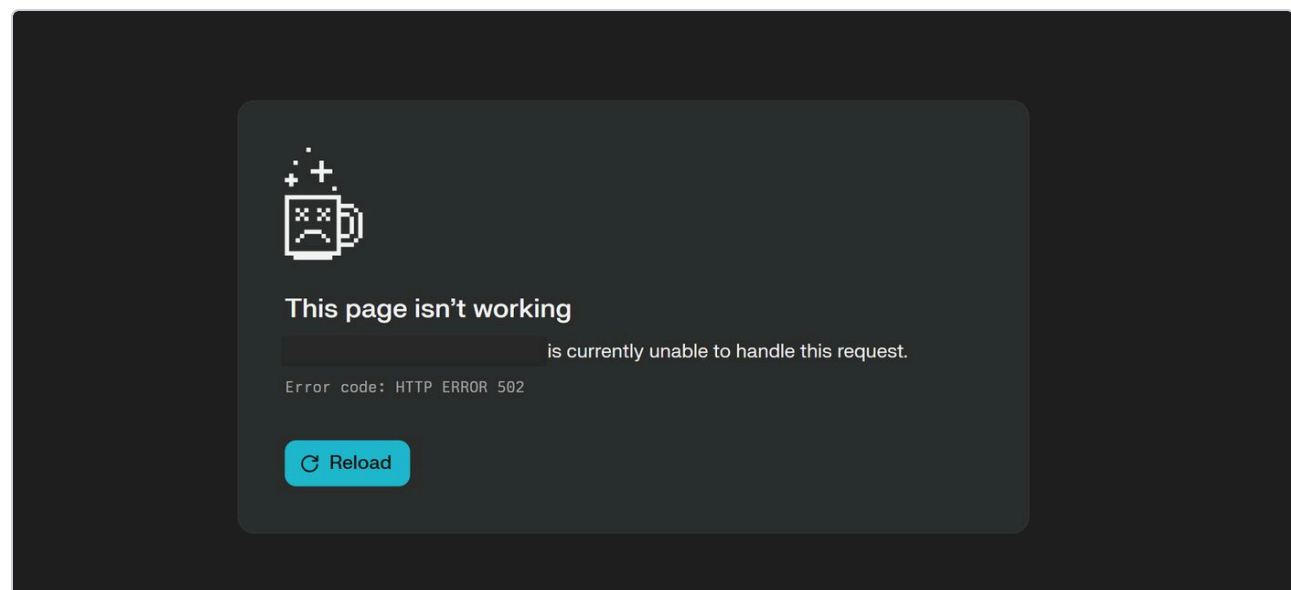


Figure 36. The same expected 502 as the first build: Caddy and its certificate came up automatically.

It built itself with no manual steps. Then I removed it:

```
terraform destroy # removes the test instance and its security group
```

My live server is still the hand-built one, and Terraform doesn't manage it. The difference is that if it ever breaks, I can rebuild it with the same locked-down settings in about five minutes.

Part 9: A private AI assistant with Ollama

The last addition was for productivity rather than security. I wanted AI features in Nextcloud, like chat, summaries, and drafting, but I didn't want my files going to an outside AI service. So the model runs on my own laptop.

Ollama runs on the Surface Book 3, the same machine that hosts the TrueNAS VM. The EC2 instance only has 2 GB of RAM, which isn't enough for a model. I used `llama3.2:3b`, a small model that runs on CPU alone. Nextcloud on EC2 reaches it over Tailscale, so prompts never leave my tailnet.

```
winget install Ollama.Ollama
ollama pull llama3.2:3b
[Environment]::SetEnvironmentVariable("OLLAMA_HOST", "0.0.0.0:11434", "User")
```

The first test failed with "Couldn't connect to server", even from the laptop itself. `netstat` showed why:

```
TCP      127.0.0.1:11434      0.0.0.0:0           LISTENING
```

The setting was saved, but Ollama had started before I set it, so it was still listening only on localhost. After a full restart, `netstat` showed it listening on every interface:

```
TCP      [::]:11434          [::]:0              LISTENING
```

Then EC2 could see the model:

```
ubuntu@cloudhub:~$ curl http://<surface-tailscale-ip>:11434/v1/models
{"object": "list", "data": [{"id": "llama3.2:3b", "object": "model", ...}]}
```

A Windows Firewall rule allows port 11434 only from the CloudHUB server's Tailscale address:

```
PS> Get-NetFirewallRule | Where-Object { $_.DisplayName -like "*ollama*" -and $_.Enabled -eq "True" } |  
    ForEach-Object { [pscustomobject]{ Name=$_.DisplayName; Action=$_.Action; Remote=(($_ | Get-NetFirewallAddressFilter).RemoteAddress) } }
```

Name	Action	Remote
-----	-----	-----
Ollama - CloudHUB only	Allow	<cloudhub-tailscale-ip>

In Nextcloud, I installed the **Nextcloud Assistant** and **OpenAI and LocalAI integration** apps. Then I set the service URL to the laptop's Tailscale address with `/v1`, chose Chat completions, and selected `llama3.2:3b`. The first chat came back in about a minute.

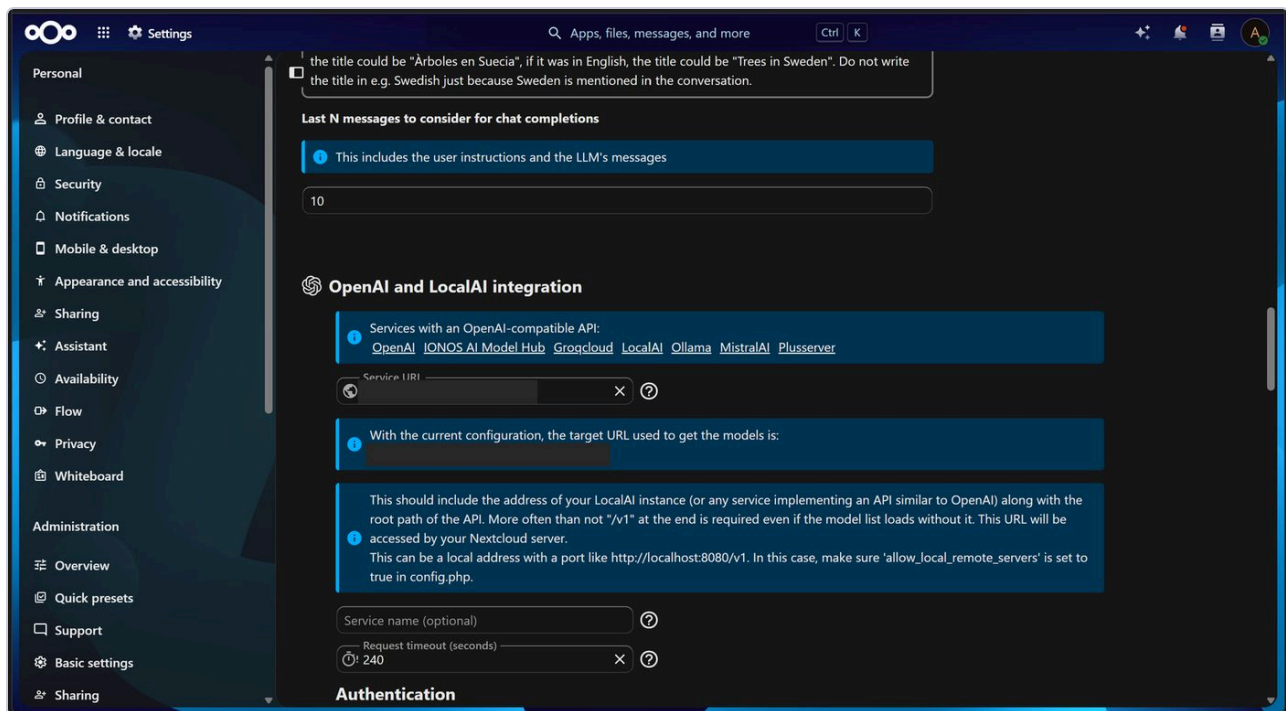


Figure 37. The service URL points at the laptop's Tailscale address with `/v1` (redacted).

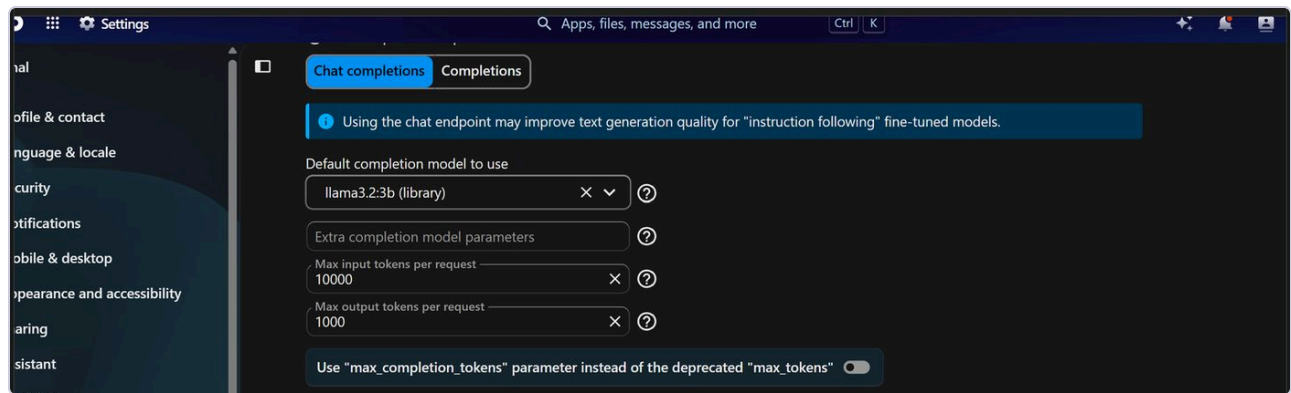


Figure 38. Once the model showed up in this dropdown, I knew Nextcloud could reach Ollama.

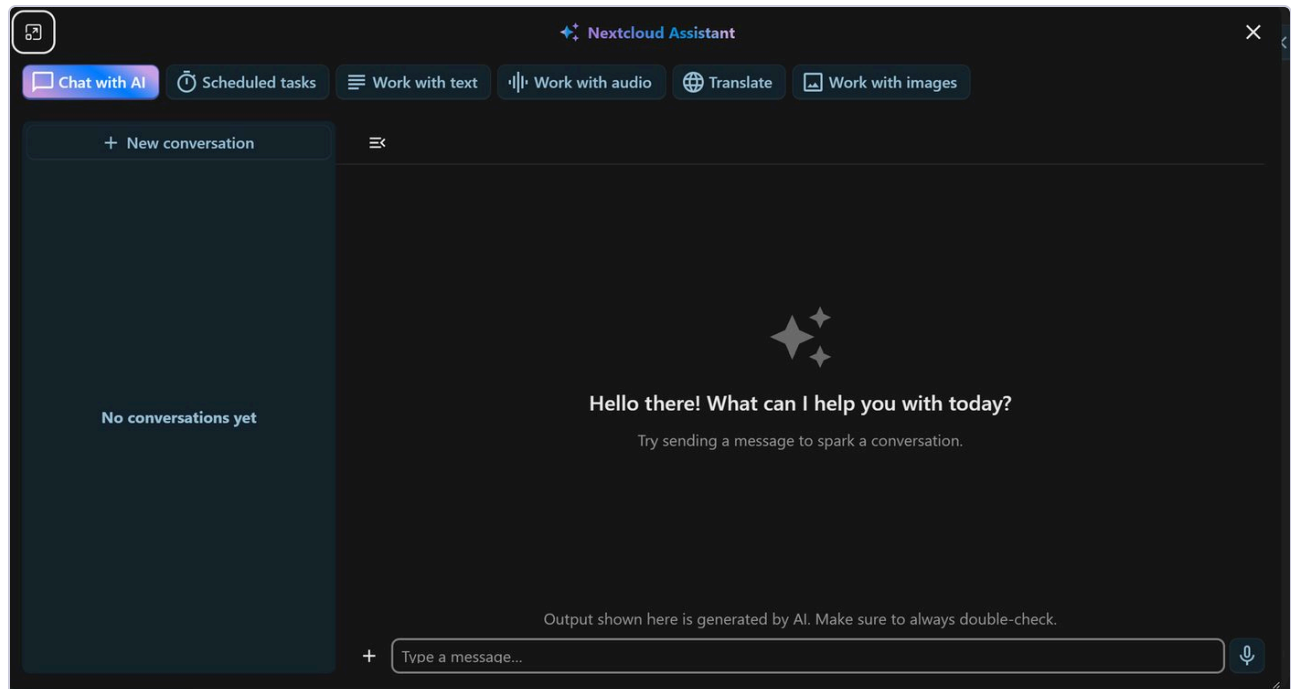


Figure 39. The Assistant, ready for its first prompt.

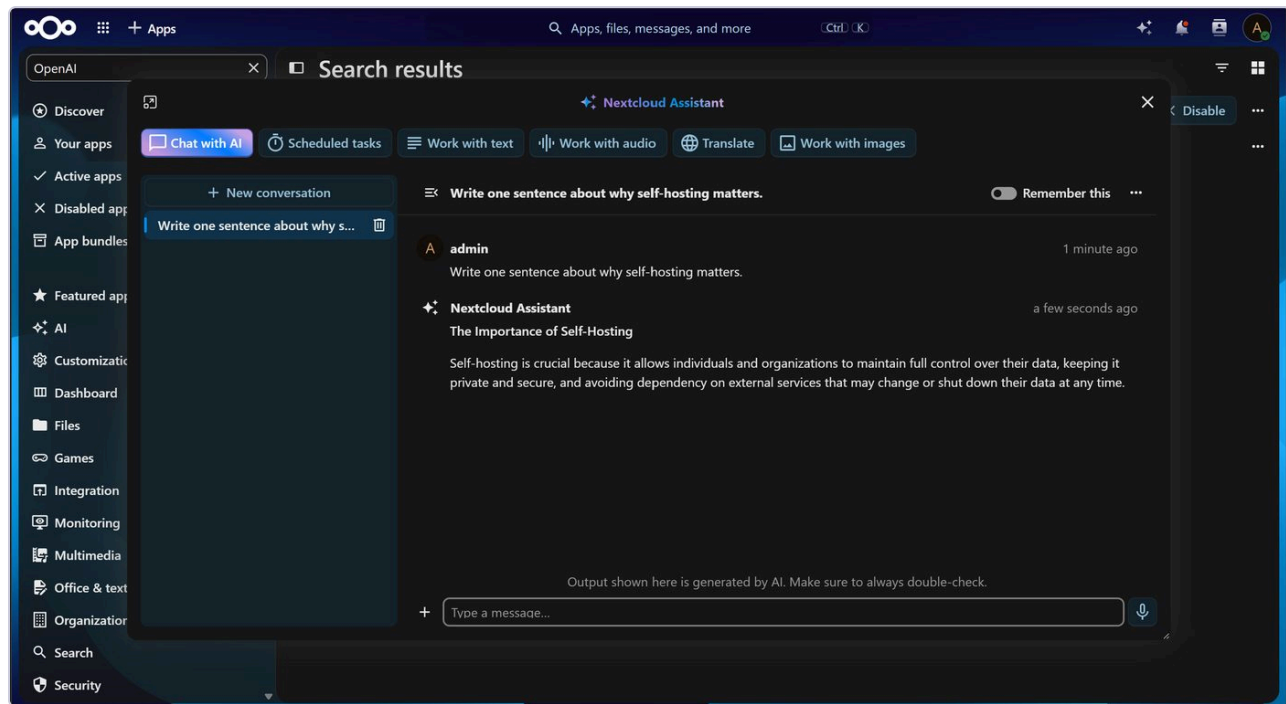


Figure 40. A Nextcloud chat, answered by a model running on my own laptop.

The trade-offs: the Assistant only works while the laptop is awake, replies take seconds to minutes on CPU, and a 3B model is weaker than the big cloud models. For summaries and first drafts, it's enough.

Part 10: A Discord bot for quick actions

I spend a lot of time in Discord, so I wanted a few CloudHUB actions there: drop a file into Nextcloud, ask the private model a question, summarize a document, and check whether everything is healthy. The bot is a small Python program (`discord.py`, `aihttp` and `pypdf`) that runs in Docker on the same EC2 instance.

Design decisions:

- **Outbound only.** The bot connects out to Discord and reaches Nextcloud and Ollama over Tailscale, so the security group stays at zero inbound rules.
- **Its own Nextcloud account.** The bot logs in as a dedicated `discord-bot` user with an app password, and only sees the one folder I share with it.
- **Locked-down container.** Non-root user, read-only filesystem, all Linux capabilities dropped, a 128 MB memory cap and no published ports.
- **Only me.** Commands work only for the Discord user IDs on an allow list, and only in the CloudHUB channel.

Command	What it does
<code>/upload</code>	Saves a Discord attachment into the <code>Discord Inbox</code> folder in Nextcloud over WebDAV
<code>/ask</code>	Sends a prompt to the private Ollama model
<code>/summarize</code>	Summarizes a text, Markdown, PDF or Word file from Nextcloud, with file-name autocomplete
<code>/status</code>	Checks EC2 disk and memory, Nextcloud, Ollama and the TrueNAS pool
alerts	Runs the same checks every 15 minutes and posts only when something breaks or recovers

Setting up the IDs. Discord's Developer Mode adds a *Copy ID* option, which I used for the server, my user and the channel.

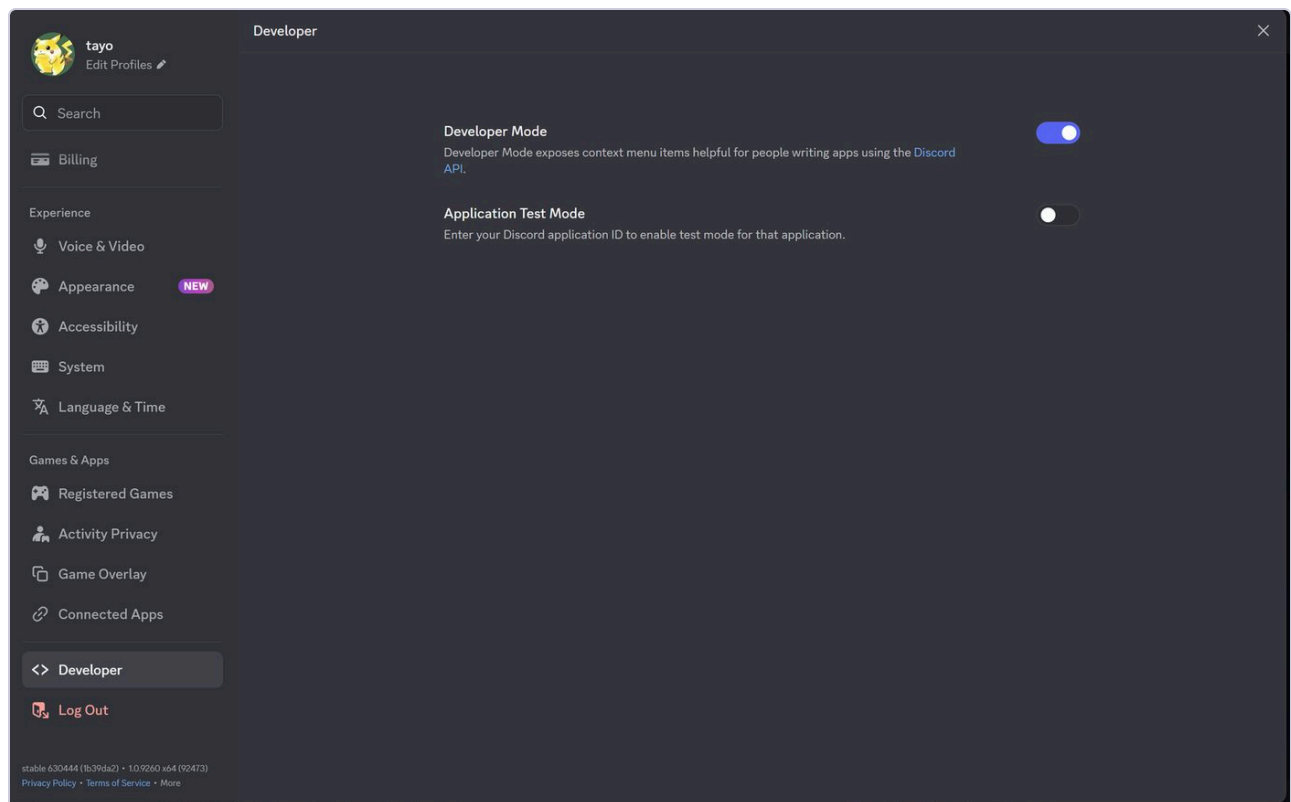


Figure 41. Developer Mode on, so server, user and channel IDs can be copied.

Creating the bot's Nextcloud account on EC2:

```
sudo docker exec -it --user www-data nextcloud-aio-nextcloud \
  php occ user:add --display-name="Discord Bot" discord-bot
```

To rename it, my first guess, `occ user:modify`, doesn't exist. The display name is a user setting:

```
sudo docker exec --user www-data nextcloud-aio-nextcloud \
  php occ user:setting discord-bot settings display_name "CloudHUB Bot"
```

Then, as admin, I created a `Discord Inbox` folder and shared it with the bot with edit rights, so uploads land in my own files. From a private browser window logged in as the bot, I created an app password for it.

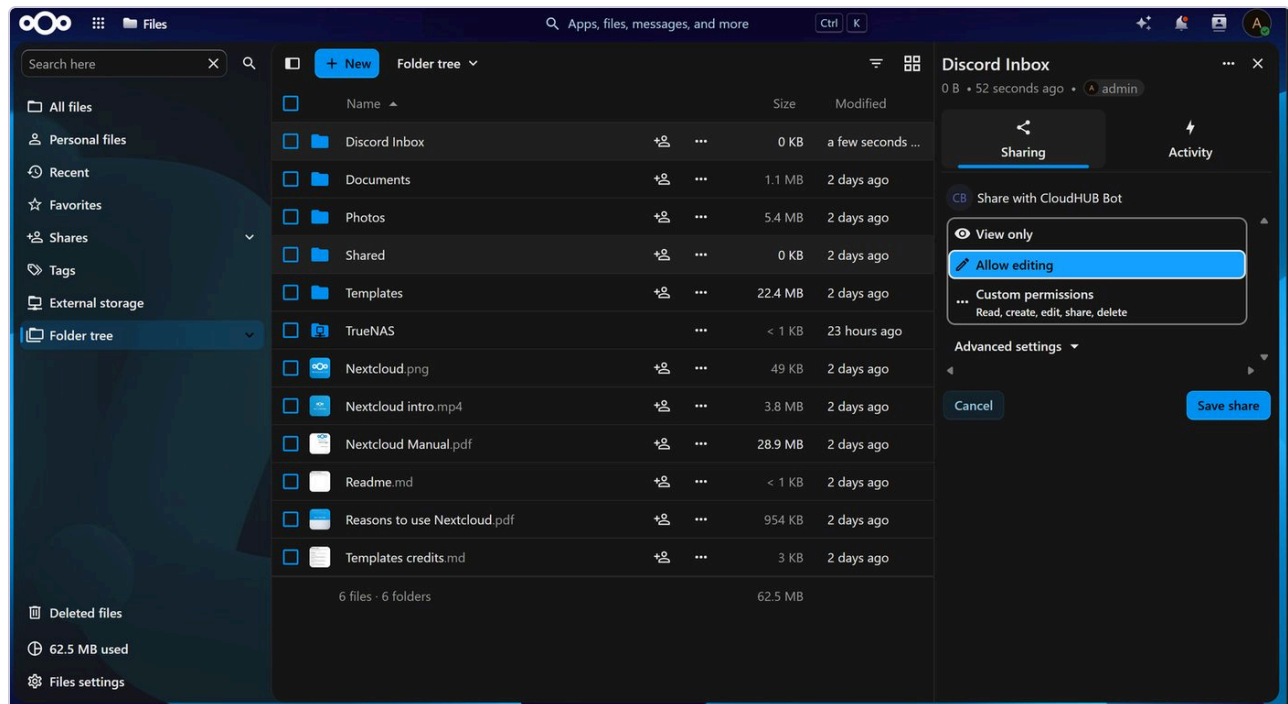


Figure 42. Sharing the Discord Inbox folder with CloudHUB Bot, with editing allowed.

Deploying it:

```
scp -r discord-bot cloudhub:~/
ssh cloudhub
sudo mkdir -p /opt/cloudhub-bot/probe
cd ~/discord-bot
cp .env.example .env && nano .env
chmod 600 .env
sudo docker compose up -d --build
sudo docker logs -f cloudhub-bot
```

The first start failed with `Improper token has been passed`. A quick check that printed only the token's length and number of dots, never the token itself, explained it:

length: 19 | dots: 0

A real bot token is about 70 characters with two dots. Nineteen digits is the **Application ID** from the General Information page. I reset the token on the **Bot** tab, pasted that into `.env`, and the bot came online as `CloudHUB#0933`.

Testing every command. `/ask` answered from the local model, and `/status` came back all green:

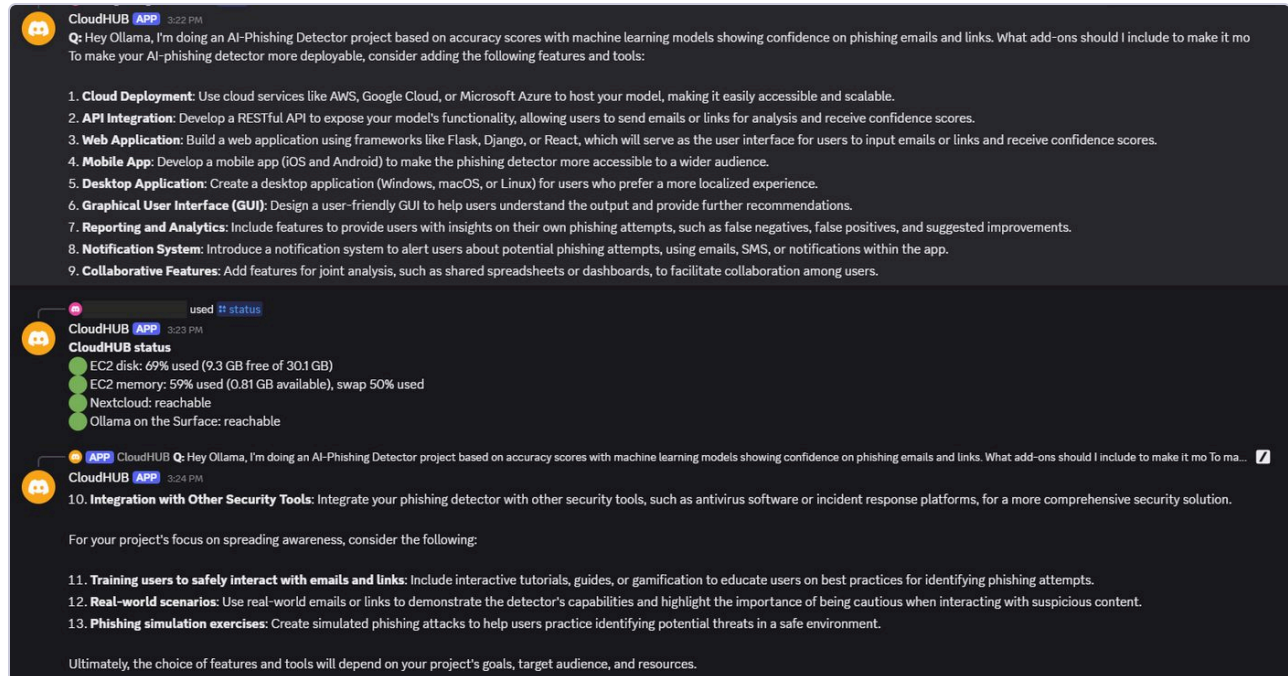


Figure 43. `/ask` answered by Ollama, and `/status` showing EC2 disk at 69%, memory at 59% with 0.81 GB available, and Nextcloud and Ollama reachable.

That status check also answered whether the t3.small needed an upgrade. With Nextcloud and the bot both running, there was still 0.81 GB of memory available, so I stayed on the 2 GB instance.

`/upload` worked, but `/summarize` didn't at first. The first try was a Word file with a typed-in path:

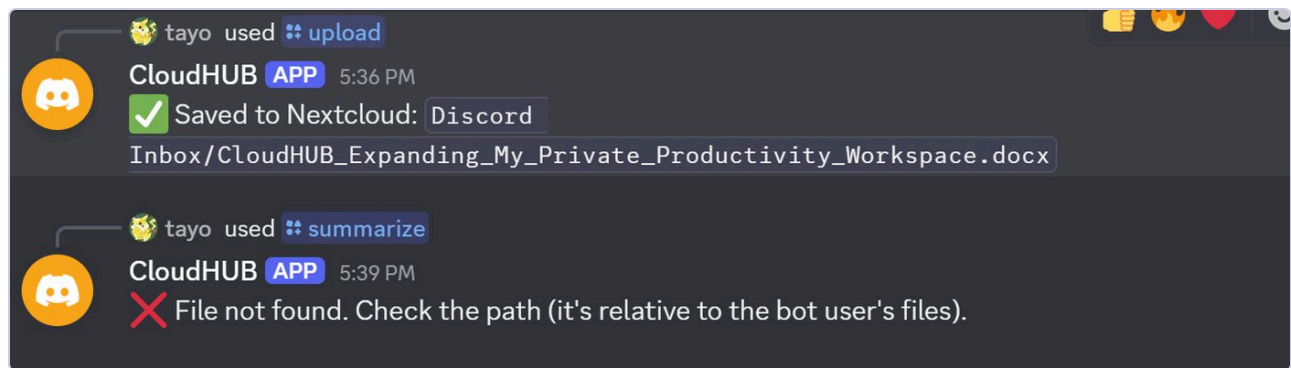


Figure 44. The upload saved, but the summary failed with File not found.

Two things were wrong: a typed path is easy to get slightly wrong, and the bot couldn't read `.docx` files yet. I added file-name autocomplete from the inbox, so you pick the file instead of typing it, and Word support that reads the text out of the document. A Markdown copy of the same post then summarized correctly:

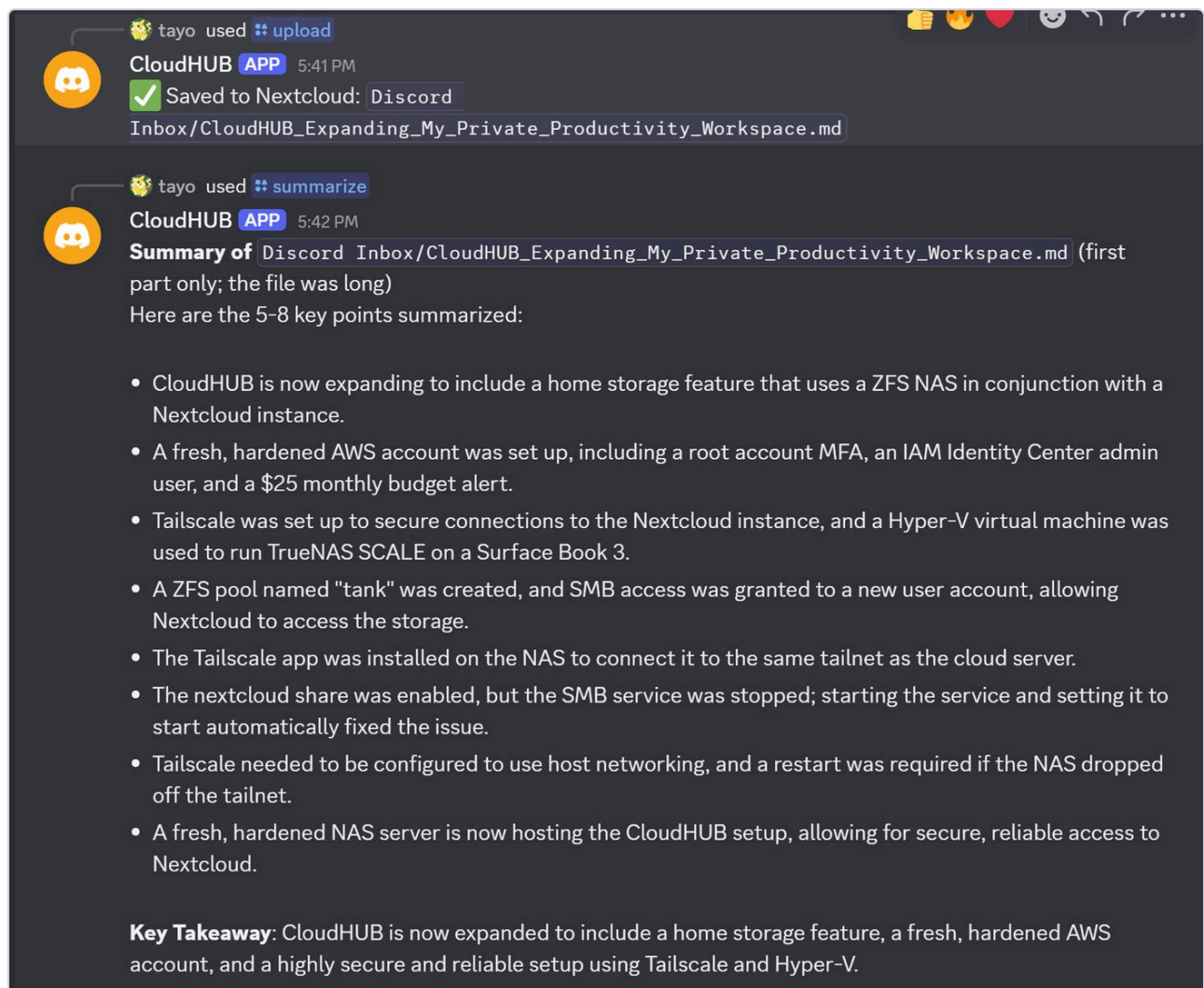


Figure 45. Uploading a Markdown file and summarizing it with the local model.

The summary was mostly right, but not completely. It claimed a "fresh, hardened NAS server is now hosting the CloudHUB setup," which isn't what the post says. A 3B model summarizes loosely, so I treat its output as a starting point, not a source of truth.

The bot has 25 offline tests that run against a fake Nextcloud, Ollama and TrueNAS:

```
python -m pytest -q
```

Part 11: Storage alerts from the home host

The bot on EC2 can't see the laptop's own drives, so a small PowerShell script runs on the laptop instead. Every 30 minutes it checks each drive's free space and whether the TrueNAS VM is running. It posts to the CloudHUB channel through a Discord webhook only when something changes: a drive goes over 85% used, the VM stops, or either one recovers.

Keeping the webhook secret. Anyone with a webhook URL can post to the channel, so the URL lives in a file that only Administrators and SYSTEM can read, not in the script:

```
$dir = "$env:ProgramData\CloudHUB"
New-Item -ItemType Directory -Path $dir -Force | Out-Null
Set-Content "$dir\webhook.txt" "<webhook-url>"
icacls $dir /inheritance:r /grant:r "Administrators:(OI)(CI)F" "SYSTEM:(OI)(CI)F" | Out-Null
```

The first run said the script file didn't exist. It did exist; I had run PowerShell as my normal user, and the folder's new permissions locked that user out. Running PowerShell as Administrator fixed it, and the install steps now check for admin rights first.

The second run was rejected by Discord:

```
{"message": "The request body contains invalid JSON.", "code": 50109}
```

Windows PowerShell 5.1 reads a `.ps1` file without a byte-order mark as the system code page, so the emoji in the message turned into garbage, and it also doesn't send a string body as UTF-8. I rewrote the script to be plain ASCII, building the emoji from code points, and to send the JSON as UTF-8 bytes:

```
$EmojiTest = [char]::ConvertFromUtf32(0x1F9EA)
...
$json = @{ content = $Text } | ConvertTo-Json
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)
Invoke-RestMethod -Uri $webhook -Method Post -ContentType 'application/json; charset=utf-8' -Body $bytes
```

One more trap: I re-ran my old install block, which copied the old script back from Downloads by its file name. The final install block only installs a copy that contains the fix:

```
Installing Watch-SurfaceHealth.ps1
Test message sent.
```

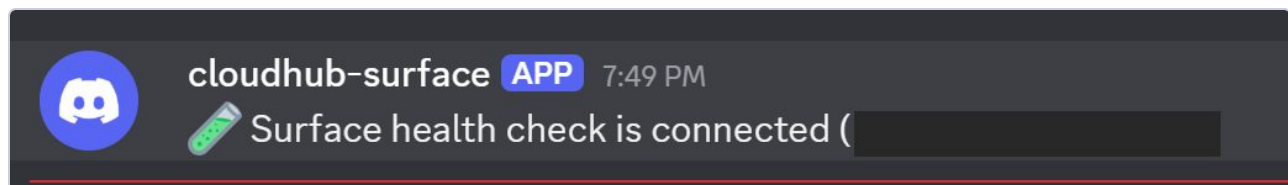


Figure 46. The test message from the laptop's health check (hostname redacted).

Scheduling it as SYSTEM every 30 minutes:

```
$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$script`""
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-TimeSpan -Minutes 30)
Register-ScheduledTask -TaskName "CloudHUB Surface Health" -Action $action -Trigger $trigger -User "SYSTEM" -RunLevel Highest -Force
Start-ScheduledTask -TaskName "CloudHUB Surface Health"
```

```
Scheduled and working. Last run: 10/08/2026 20:22:25
```

The task's last result was 0, so it ran cleanly. The first scheduled run posts nothing, because nothing has changed yet.

What I learned

1. **Verify what you got, not what you planned.** The port scan caught a security group I assumed was locked down.

2. **Build pre-flight checks into your scripts.** The disk-space check stopped a bad VM build before anything was created.
3. **Read errors literally.** NXDOMAIN was a DNS problem, and a 502 meant the proxy was up with nothing behind it yet.
4. **"Running" isn't the same as healthy.** Memory pressure made the instance stop responding while AWS still showed it running.
5. **Apply least privilege on both ends.** A dedicated SMB user on the NAS, a mount only the admin can see, and a separate account and app password for the bot.
6. **Code is how you keep the fixes.** Terraform makes the zero-inbound security group the default, not something I have to remember.
7. **A service has to listen where you expect it to.** Ollama was running fine, just on the wrong interface. `netstat` answered in one line what guessing couldn't.
8. **Check what a secret looks like without printing it.** Its length and format showed I had pasted the Application ID, not the bot token.
9. **Encoding matters on Windows PowerShell 5.1.** Keep scripts ASCII and send JSON as UTF-8 bytes.
10. **A small model is a starting point.** Its summaries are useful, but they need a quick check against the source.

Trade-offs

- File transfers between AWS and home are limited by my home upload speed.
- The NAS, the AI assistant and the storage alerts only work while the laptop is awake and online.
- The pool is a single virtual disk with no redundancy, so the snapshots are not a backup on their own.
- Anything the bot posts (answers, summaries, alerts) is stored on Discord's servers, so sensitive documents stay in the Nextcloud Assistant.

Stack

Nextcloud AIO · AWS EC2 (t3.small, Ubuntu 24.04) · Docker · Tailscale · Caddy · TrueNAS SCALE · ZFS · SMB · Hyper-V · Terraform · AWS IAM Identity Center · Ollama · Python (discord.py, aiohttp) · PowerShell · Discord webhooks

Code: github.com/by-tayo/CloudHUB

Appendix: Setup reference

The full step-by-step setup, with the exact commands, for anyone who wants to rebuild CloudHUB.

Prerequisites

- AWS account with EC2 access
- Tailscale account
- Ubuntu 24.04 LTS EC2 instance (t3.small minimum)
- Docker Engine installed

AWS Configurations

Step 1: AWS EC2 Setup

EC2 Instance Configuration:

Setting	Value
Name	<code>your-instance</code>
AMI	Ubuntu Server 24.04 LTS (HVM)
AMI ID	Latest Ubuntu 24.04 LTS AMI for your region (AMI IDs are region-specific)
Instance Type	<code>t3.small</code> (2 vCPU, 2 GB RAM)
Region	us-east-2 (Ohio)
Storage	30 GB gp3
Key Pair	<code>your-instance.pem</code>

Security Group (Tailscale-only)

CloudHUB is reached only over the tailnet, and Caddy uses Tailscale-issued certificates, so no web ports need to be open to the internet.

Port	Protocol	Source	Purpose
22	TCP	My IP	SSH admin access during setup (remove once Tailscale SSH works)
8080	TCP	My IP	Nextcloud AIO setup interface (remove after initial setup)
41641	UDP	Anywhere	<i>Optional:</i> lets Tailscale make direct peer connections instead of relaying through DERP

No rule for 80, 443, or 8443: all Nextcloud traffic arrives through the Tailscale tunnel. Outbound rules stay at the default (all traffic) so the instance can reach Tailscale, Docker Hub, and package mirrors.

SSH Into the Instance

```
ssh -i ~/.ssh/[your-key].pem ubuntu@[your-ec2-ip]
```

```
# Update system
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget
sudo reboot
```

 After reboot, reconnect using the same SSH command.

Quick Start

1. Install Docker:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
```

2. Install and configure Tailscale:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
sudo tailscale cert your-machine.tail1234ab.ts.net
```

3. Configure Caddy:

```
your-machine.tail1234ab.ts.net {
  reverse_proxy localhost:11000
  tls /var/lib/tailscale/certs/your-machine.tail1234ab.ts.net.crt \
    /var/lib/tailscale/certs/your-machine.tail1234ab.ts.net.key
}
```

4. Run Nextcloud AIO:

```
docker run \
  --sig-proxy=false \
  --name nextcloud-aio-mastercontainer \
  --restart always \
  --publish 8080:8080 \
  --env APACHE_PORT=11000 \
  --env APACHE_IP_BINDING=127.0.0.1 \
  --env SKIP_DOMAIN_VALIDATION=true \
  --volume nextcloud_aio_mastercontainer:/mnt/docker-aio-config \
  --volume /var/run/docker.sock:/var/run/docker.sock:ro \
  nextcloud/all-in-one:latest
```

Infrastructure as Code (Terraform)

The [terraform/](#) folder recreates the EC2 side of CloudHUB from scratch: the latest Ubuntu 24.04 AMI, a security group with **no inbound rules** (all outbound), an encrypted 30 GB gp3 volume, IMDSv2-only metadata, and no SSH key pair — admin access is Tailscale SSH only. A cloud-init script then installs swap, Docker, Tailscale (with `--ssh`), Caddy with auto-renewing Tailscale certificates, and the Nextcloud AIO master container.

File	Purpose
<code>versions.tf</code>	Terraform + AWS provider versions, default tags
<code>variables.tf</code>	Region, instance type, disk/swap size, Tailscale auth key (sensitive)
<code>main.tf</code>	AMI lookup, security group, EC2 instance
<code>cloud-init.yaml.tftpl</code>	Bootstrap script, logged to <code>/var/log/cloudhub-setup.log</code>
<code>outputs.tf</code>	Instance ID, IPs, next steps
<code>terraform.tfvars.example</code>	Copy to <code>terraform.tfvars</code> (git-ignored) and fill in

```
cd terraform
cp terraform.tfvars.example terraform.tfvars # add a one-off Tailscale auth key
terraform init
terraform plan
terraform apply
# ...test it...
terraform destroy
```

The Tailscale auth key ends up in instance user data, so use a **single-use, ephemeral, short-expiry** key and never commit `terraform.tfvars`.

Authenticate with SSO, not access keys:

```
aws configure sso # session name: cloudhub-sso, region: us-east-2, profile: cloudhub
aws sts get-caller-identity --profile cloudhub
aws sso login --profile cloudhub # when the session expires
```

Verified test run: `apply` created the security group and the instance. Afterward, the security group had no inbound rules, the instance showed Connected (Ephemeral, SSH) in Tailscale, `cloud-init status` returned `done`, the setup log ended with `CloudHUB setup finished`, a 2 GB swapfile was active, and the AIO login page loaded on port 8080 over Tailscale. Then `destroy` removed both resources.

Nextcloud Assistant with a Local LLM (Ollama)

Ollama runs on the Surface Book 3 (CPU only), and Nextcloud on EC2 reaches it over Tailscale. Prompts and file text never leave the tailnet.

1. Install Ollama and pull a small model (Surface, PowerShell)

```
if (Get-Command ollama -ErrorAction SilentlyContinue) { ollama --version } else { winget install Ollama.Ollama }
ollama pull llama3.2:3b
ollama run llama3.2:3b "In one sentence, what is a NAS?"
```

2. Listen beyond localhost, and allow only CloudHUB through the firewall

```
[Environment]::SetEnvironmentVariable("OLLAMA_HOST", "0.0.0.0:11434", "User")
$env:OLLAMA_HOST = "0.0.0.0:11434"
Get-Process | Where-Object { $_.ProcessName -like "ollama*" } | Stop-Process -Force # full restart is required
$app = Get-ChildItem "$env:LOCALAPPDATA\Programs\Ollama" -Filter "ollama app.exe" -ErrorAction SilentlyContinue | Select-Object -First 1
if ($app) { Start-Process $app.FullName } else { Start-Process ollama -ArgumentList "serve" -WindowStyle Minimized }
netstat -ano | findstr ":11434" # expect 0.0.0.0 or [::], not 127.0.0.1

# Administrator PowerShell: allow port 11434 only from the CloudHUB EC2 Tailscale IP
New-NetFirewallRule -DisplayName "Ollama - CloudHUB only" -Direction Inbound -Protocol TCP -LocalPort 11434 -RemoteAddress <cloudhub-tailscale-ip> -Action Allow
```

3. Test from EC2

```
curl http://<surface-tailscale-ip>:11434/v1/models # should list llama3.2:3b
```

4. Install the Nextcloud apps

```
for app in assistant integration_openai; do
  sudo docker exec --user www-data nextcloud-aio-nextcloud php occ app:install "$app" \
  || sudo docker exec --user www-data nextcloud-aio-nextcloud php occ app:enable "$app"
done
```

5. Configure in *Administration settings* → *Artificial intelligence* → *OpenAI and LocalAI integration*:

Setting	Value
Service URL	<code>http://<surface-tailscale-ip>:11434/v1</code>
API key	blank
Endpoint	Chat completions
Default model	<code>llama3.2:3b</code>
Request timeout	240 s (CPU inference is slow)

Then point the text task providers (Free text to text, Summarize, Generate headline) at *OpenAI and LocalAI integration*.

If the model list stays empty, Nextcloud may be blocking the 100.x Tailscale address as a "local" address. `occ config:system:set allow_local_remote_servers --value=true --type=boolean` allows it; that relaxes Nextcloud's protection against requests to internal addresses, so only use it on an instance that, like this one, is reachable only over the tailnet.

Trade-offs: the Assistant works only while the Surface is awake and online; replies take seconds to minutes on CPU; Nextcloud only officially tests the Assistant against OpenAI's models.

TrueNAS SCALE on Hyper-V (Hybrid Storage)

Prerequisites - Windows 10/11 **Pro, Enterprise, or Education** (Hyper-V is not available on Home) - 16 GB+ RAM on the host (the VM gets 16 GB fixed) - ~120 GB free disk space - TrueNAS SCALE / Community Edition ISO from truenas.com, saved to your **Downloads** folder - The CloudHUB EC2 host already on your Tailscale tailnet

1. Keep the host awake

A sleeping host pauses the VM and Nextcloud loses the share. - **Settings → System → Power & battery → Screen, sleep & hibernate timeouts** → *Make my device sleep after* (plugged in): **Never** - **Lid & power button controls** → *Closing the lid will make my PC* (plugged in): **Do Nothing**

2. Check / enable Hyper-V (PowerShell as Administrator):

```
& {
    $os = (Get-CimInstance Win32_OperatingSystem).Caption
    Write-Host "Windows edition: $os"
    if ($os -match "Home") {
        Write-Host "Home edition: Hyper-V is not available. Stop here." -ForegroundColor Red
        return
    }
    $hv = Get-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All
    if ($hv.State -eq "Enabled") {
        Write-Host "Hyper-V is already enabled. Skip the restart." -ForegroundColor Green
    } else {
        Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All -All -NoRestart | Out-Null
        Write-Host "Hyper-V enabled. Restart Windows, then continue." -ForegroundColor Yellow
    }
}
```

3. Create the VM (PowerShell as Administrator):

Pre-flight checks run first — the script stops before creating anything if the ISO is missing, the VM name is taken, the switch doesn't exist, or there isn't enough free disk space.

```
& {
    $VMName = "TrueNAS"
    $VMPath = "C:\HyperV\$VMName"
    $MemGB = 16      # fixed RAM; ZFS uses it for caching
    $CPUs = 4
    $BootGB = 32     # TrueNAS OS disk
    $DataGB = 64     # storage pool disk (dynamic: only grows as you fill it)

    $iso = Get-ChildItem "$env:USERPROFILE\Downloads" -Filter "TrueNAS*.iso" -ErrorAction SilentlyContinue |
        Sort-Object LastWriteTime -Descending | Select-Object -First 1
    if (-not $iso) { Write-Host "No TrueNAS ISO found in Downloads. Download it, then rerun." -ForegroundColor Red; return }
    Write-Host "Using ISO: $($iso.Name)"

    if (Get-VM -Name $VMName -ErrorAction SilentlyContinue) {
        Write-Host "A VM named '$VMName' already exists. Stopping so nothing gets overwritten." -ForegroundColor Yellow; return
    }
    if (-not (Get-VMSwitch -Name "Default Switch" -ErrorAction SilentlyContinue)) {
        Write-Host "Hyper-V 'Default Switch' not found. Did you restart after enabling Hyper-V?" -ForegroundColor Red; return
    }
    $freeGB = [math]::Round((Get-PSDrive C).Free / 1GB)
    if ($freeGB -lt ($BootGB + $DataGB + 20)) {
        Write-Host "Only $freeGB GB free on C:. Lower `$DataGB and rerun." -ForegroundColor Red; return
    }

    New-Item -ItemType Directory -Path $VMPath -Force | Out-Null
    New-VM -Name $VMName -Generation 2 -MemoryStartupBytes ($MemGB * 1GB) -Path $VMPath `
        -NewVHDPATH "$VMPath\boot.vhdx" -NewVHDSIZEBytes ($BootGB * 1GB) -SwitchName "Default Switch" | Out-Null
    Set-VMemory -VMName $VMName -DynamicMemoryEnabled $false # TrueNAS needs fixed memory
    Set-VMProcessor -VMName $VMName -Count $CPUs
    Set-VM -Name $VMName -CheckpointType Disabled # use ZFS snapshots instead
    New-VHD -Path "$VMPath\data.vhdx" -SizeBytes ($DataGB * 1GB) -Dynamic | Out-Null
    Add-VMHardDiskDrive -VMName $VMName -Path "$VMPath\data.vhdx"
    $dvd = Add-VMVDDrive -VMName $VMName -Path $iso.FullName -Passthru
    Set-VMFirmware -VMName $VMName -EnableSecureBoot Off -FirstBootDevice $dvd

    Write-Host "VM '$VMName' created: $MemGB GB RAM, $CPUs CPUs, $BootGB GB boot disk, $DataGB GB data disk." -ForegroundColor Green
}
```

4. Install TrueNAS 1. **Hyper-V Manager** → right-click **TrueNAS** → **Connect** → **Start** 2. Press any key at "Press any key to boot from CD or DVD" (it only shows for a few seconds) 3. **Install/Upgrade** → select the **32 GB** disk only → set the `truenas_admin` password 4. **Media** → **DVD Drive** → **Eject**, then reboot 5. Open the web UI address shown on the console (e.g. `http://172.x.x.x`) in a browser on the host

5. Create the pool — **Storage** → **Create Pool** - Name: `tank` · Layout: **Stripe** · Disk: the **64 GB** data disk - If prompted, allow non-unique disk serials (normal for Hyper-V virtual disks)

6. Join the tailnet — **Apps** → **Discover Apps** → **Tailscale** (community train; choose `tank` as the apps pool)

Setting	Value
Auth Key	single-use key from Tailscale admin → Settings → Keys
Hostname	your choice
Userspace	☐ unchecked
Host Network	☒ enabled

Host networking with userspace off is what lets Nextcloud reach the SMB service on the TrueNAS host itself. In the Tailscale admin console, **Disable key expiry** on the NAS so it doesn't drop off later.

7. Dataset, user, and share 1. **Datasets** → `tank` → **Add Dataset** → name `cloudhub`, preset **SMB** 2. **Credentials** → **Users** → **Add** → `nextcloud-smb`, **SMB access only** (no shell, no TrueNAS access) 3. **Datasets** → `cloudhub` → **Permissions** → **Edit** → add **User** `nextcloud-smb` · **Modify** 4. **Shares** → **SMB** → confirm share `cloudhub` → `/mnt/tank/cloudhub` is enabled 5. **System** → **Services** → **SMB**: Running, **Start Automatically** on

8. Verify from EC2 (SSH into the CloudHUB instance):

```

NAS_IP="100.x.y.z" # <-- your TrueNAS Tailscale IP

if ! command -v tailscale >/dev/null 2>&1; then
    echo "Tailscale is not installed on this host – stop here."
else
    echo "== Tailscale ping to TrueNAS =="
    sudo tailscale ping -c 3 "$NAS_IP" || echo "Ping failed – check both machines show Connected."

    echo "== SMB port 445 =="
    if command -v nc >/dev/null 2>&1; then
        nc -zvw5 "$NAS_IP" 445 && echo "Port 445 reachable" || echo "Port 445 blocked – check Tailscale ACLs."
    else
        timeout 5 bash -c "echo > /dev/tcp/$NAS_IP/445" && echo "Port 445 reachable" || echo "Port 445 blocked – check Tailscale ACLs."
    fi
fi

```

9. Mount in Nextcloud — Apps → enable External storage support, then Administration settings → External storage:

Field	Value
Folder name	TrueNAS
External storage	SMB/CIFS
Authentication	Username and password
Host	your TrueNAS Tailscale IP (use the IP, not the MagicDNS name)
Share	cloudhub
Username / Password	nextcloud-smb credentials

A green dot next to the mount means it's connected.

10. Protect it — Data Protection → Periodic Snapshot Tasks → Add → dataset tank/cloudhub, daily, ~2 weeks retention. Make sure your home router does **not** forward port 445.

Discord bot

1. Create the Discord bot

1. Go to <https://discord.com/developers/applications> → **New Application** → name it CloudHUB.

2. **Bot** tab → **Reset Token** → copy it (this is `DISCORD_TOKEN`). Leave all *Privileged Gateway Intents* **off**.
3. **OAuth2** → **URL Generator**: scopes `bot` + `applications.commands`; bot permissions **View Channels** + **Send Messages**. Open the URL and add the bot to your server.
4. In Discord: **User Settings** → **Advanced** → **Developer Mode** on. Then right-click to **Copy ID** for your server (`GUILD_ID`), your profile (`ALLOWED_USER_IDS`), the CloudHUB category (`CATEGORY_ID`), and the channel that should receive alerts (`ALERT_CHANNEL_ID`).

2. Create the Nextcloud bot account (on EC2)

```
sudo docker exec -it --user www-data nextcloud-aio-nextcloud \
  php occ user:add --display-name="Discord Bot" discord-bot
```

Then: 1. **As admin**, create a folder named `Discord Inbox` and share it with `discord-bot` with **edit** rights. Uploads land in *your* files. 2. Log in as `discord-bot` in a private window → **Settings** → **Security** → **Create new app password** → copy it (`NC_APP_PASSWORD`).

3. Optional: a read-only TrueNAS API key

In TrueNAS, create a user (for example `discord-bot-ro`) with the **Read-Only Administrator** role. Then create an API key for that user from the account menu → **API Keys**. Set `TRUENAS_URL=https://<truenas-tailscale-ip>`. TrueNAS revokes API keys that are used over plain HTTP, so keep it on HTTPS.

4. Deploy on EC2

```
# from your laptop, in the folder that contains discord-bot/
scp -r discord-bot cloudhub:~/

# on EC2
ssh cloudhub
sudo mkdir -p /opt/cloudhub-bot/probe
cd ~/discord-bot
cp .env.example .env && nano .env      # fill in every value
chmod 600 .env
sudo docker compose up -d --build
sudo docker logs -f cloudhub-bot      # expect "Synced 4 commands" and "Logged in as CloudHUB"
```

Ollama already allows the EC2 Tailscale IP through the Surface firewall, so nothing changes there.

5. Test it in #CloudHUB

`/status` → `/ask prompt: what is ZFS?` → `/upload` a small `.md` file → `/summarize path: Discord Inbox/<file>.md`

6. Surface alerts (Administrator PowerShell on the Surface)

1. In Discord: **#CloudHUB** → **Edit Channel** → **Integrations** → **Webhooks** → **New Webhook** → copy the URL.
2. Save the webhook URL where only Administrators and SYSTEM can read it:

```
$dir = "$env:ProgramData\CloudHUB"
New-Item -ItemType Directory -Path $dir -Force | Out-Null
Set-Content "$dir\webhook.txt" "PASTE-WEBHOOK-URL-HERE"
icacls $dir /inheritance:r /grant:r "Administrators:(OI)(CI)F" "SYSTEM:(OI)(CI)F" | Out-Null
Copy-Item .\surface\Watch-SurfaceHealth.ps1 "$dir\Watch-SurfaceHealth.ps1"
& "$dir\Watch-SurfaceHealth.ps1" -Test      # a test message should appear in #CloudHUB
```

1. Run it every 30 minutes as SYSTEM:

```
$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$env:ProgramData\CloudHUB\Watch-SurfaceHealth.ps1`""
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-TimeSpan -Minutes 30)
Register-ScheduledTask -TaskName "CloudHUB Surface Health" -Action $action -Trigger $trigger -User "SYSTEM" -RunLevel Highest
```